

UNIVERSITY OF BUEA

Buea, South West Region

Cameroon

P.O. Box 63,

Tel: (237) 3332 21 34/3332 26



REPUBLIC OF CAMEROON

PEACE - WORK - FATHERLAND

Task5:

UI DESIGN AND FRONTEND IMPLEMENTATION

Instructor: **Dr Nkemeni Valery**

Course: **CEF440: Internet Programming and Mobile Programming**

By Group7

NAMES	MATRICULE
FONYUY VERENA MONYUYTA-AH	FE22A220
KENFACK DONJIO ABEL BRUNEL	FE22A380
NSONDO MIRELLE NYISEKINYI	FE22A283
TATA THECLAIRE GHALANYUY	FE22A310
UNJI STEPHEN UKU	FE22A323

2024/2025 ACADERMIC YEAR

Table of content

Catalog

Table of content	- 1 -
Introduction	- 1 -
I. App Identity	- 1 -
Branding and Purpose	- 1 -
Logo Design	- 2 -
II. Visual Design	- 2 -
UI/UX Design with Figma	- 2 -
A. Welcome Screen Design	- 3 -
B. Role Selection Screen Design	- 3 -
C. Sign-Up and login Screens	- 4 -
Verification and Two-Factor Authentication (2FA) Screens	- 6 -
D. Dashboard Designs	- 8 -
i. Lecturer Dashboard Design	- 8 -
<i>Dashboard Overview</i>	- 8 -
Main Functionalities	- 8 -
Layout Summary	- 8 -
Design Justification Using Nielsen’s Heuristics	- 8 -
Key Design Principles Applied	- 9 -
Detailed Components and Features	- 10 -
Header Bar	- 10 -
Quick Actions (Cards)	- 10 -
Active Courses List	- 10 -
Bottom Navigation	- 10 -
Modal Popups	- 10 -

Strengths and Impact	- 11 -
ii.Student Dashboard Design	- 12 -
Dashboard Overview	- 12 -
Main Functionalities	- 12 -
Layout Summary	- 12 -
Responsive Design	- 12 -
Design Justification Using Nielsen’s Heuristics	- 12 -
Key Design Principles Applied	- 13 -
Detailed Components and Features	- 13 -
Header Bar	- 13 -
Welcome Card	- 14 -
Quick Stats	- 14 -
Today’s Classes List	- 14 -
Bottom Navigation	- 14 -
Strengths and Impact	- 14 -
iii.Admin Dashboard Design and Implementation	- 16 -
Dashboard Overview	- 16 -
Main Functionalities	- 16 -
Layout Summary	- 16 -
Responsive Design	- 16 -
Design Justification Using Nielsen’s Heuristics	- 17 -
Key Design Principles Applied	- 17 -
Detailed Components and Features	- 18 -
Header Bar	- 18 -
Quick Stats	- 18 -
Quick Actions (Cards)	- 18 -
Bottom Navigation	- 18 -

Strengths and Impact	- 18 -
Challenges Faced	- 19 -
Screenshots	- 19 -
Frontend Implementation	- 20 -
Technology Stack	- 20 -
Folder Structure	- 20 -
o features/: Feature-specific code.	- 20 -
o main.dart: Entry point of the app.	- 21 -
File Usage and Implementation Details	- 21 -
Welcome Screen (welcome_screen.dart)	- 21 -
Sign-Up Screens	- 22 -
Verification and Success Screens	- 24 -
Dashboards and Role-Specific Functionalities	- 24 -
Login Screen (login_screen.dart)	- 28 -
Main and App Dart Files	- 28 -
Pubspec.yaml Configuration	- 29 -
Testing and Deployment	- 30 -
Frontend Implementation	- 32 -
Software Tools and Libraries	- 35 -
Software Tools and Their Functions	- 35 -
Library Rules and Usage Guidelines	- 35 -
Conclusion	- 36 -
References	- 37 -

Introduction

The AttendEase application is a mobile-based attendance management system designed to streamline the process of tracking attendance for educational institutions using advanced technologies such as geofencing and facial recognition. This report provides a comprehensive overview of the app's identity, design process, and frontend implementation. The project leverages Flutter for cross-platform development, Figma for UI/UX design, and an Android emulator for testing and deployment. The app targets three primary user roles: students, lecturers, and administrators, each with tailored dashboards and functionalities.

The report is structured as follows: Section 2 discusses the app identity, including branding and logo design. Section 3 details the design process using Figma. Section 4 covers the frontend implementation in Flutter, including the folder structure and file usage. Section 5 lists the software tools used and their roles in the project. Finally, Section 6 concludes the report with key insights and future considerations.

I. App Identity

Branding and Purpose

AttendEase is branded as a modern, efficient, and user-friendly attendance management solution. The name "AttendEase" reflects the core objective of simplifying attendance tracking through innovative technology. The app's tagline, "Smart attendance management with facial recognition and geofencing technology," emphasizes its reliance on cutting-edge features to ensure accuracy and convenience.

The primary color scheme includes shades of blue and purple, as seen in the gradient background of the welcome screen (`welcome_screen.dart`). These colors convey trust, professionalism, and modernity, aligning with the app's educational context. Secondary colors, such as success green and error red, are used for status indicators across various screens, ensuring intuitive user feedback.

Logo Design

The AttendEase logo encapsulates the app's core functionalities. It features a stylized human head with a geofencing marker above it, symbolizing location-based attendance tracking. A smartphone icon alongside the head represents the facial recognition feature, while the gradient circular background in blue and purple ties the design to the app's color scheme. The text "AttendEase" is written in a bold, modern font, with "MODERN APP" in smaller text below, reinforcing the app's contemporary approach.

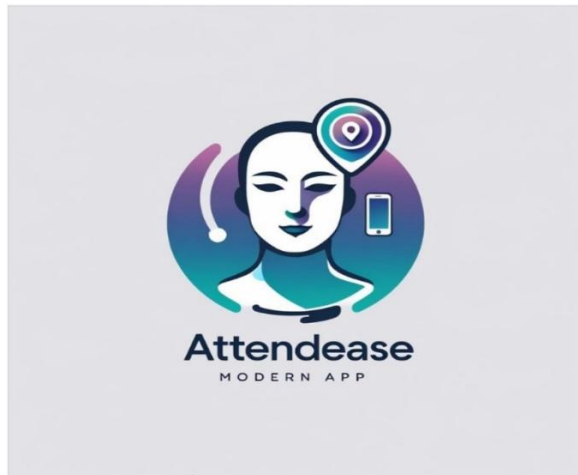


Figure 1: AttendEase logo highlighting geofencing and facial recognition elements.

II. Visual Design

UI/UX Design with Figma

The user interface (UI) and user experience (UX) design for AttendEase were created using Figma, a collaborative design tool that allowed for iterative prototyping and team feedback. Figma was chosen for its ability to create high-fidelity mockups, design systems, and interactive prototypes, ensuring a seamless transition from design to development.

A. Welcome Screen Design

The welcome screen design focuses on introducing the app's purpose and guiding users to either sign up or log in. The gradient background (blue to purple) creates a visually appealing entry point, while the logo and tagline are prominently displayed. Two buttons, "Sign Up" and "Login," are styled with the app's primary colors, ensuring consistency. There is a transition in home page changing its content like a quick description of the app

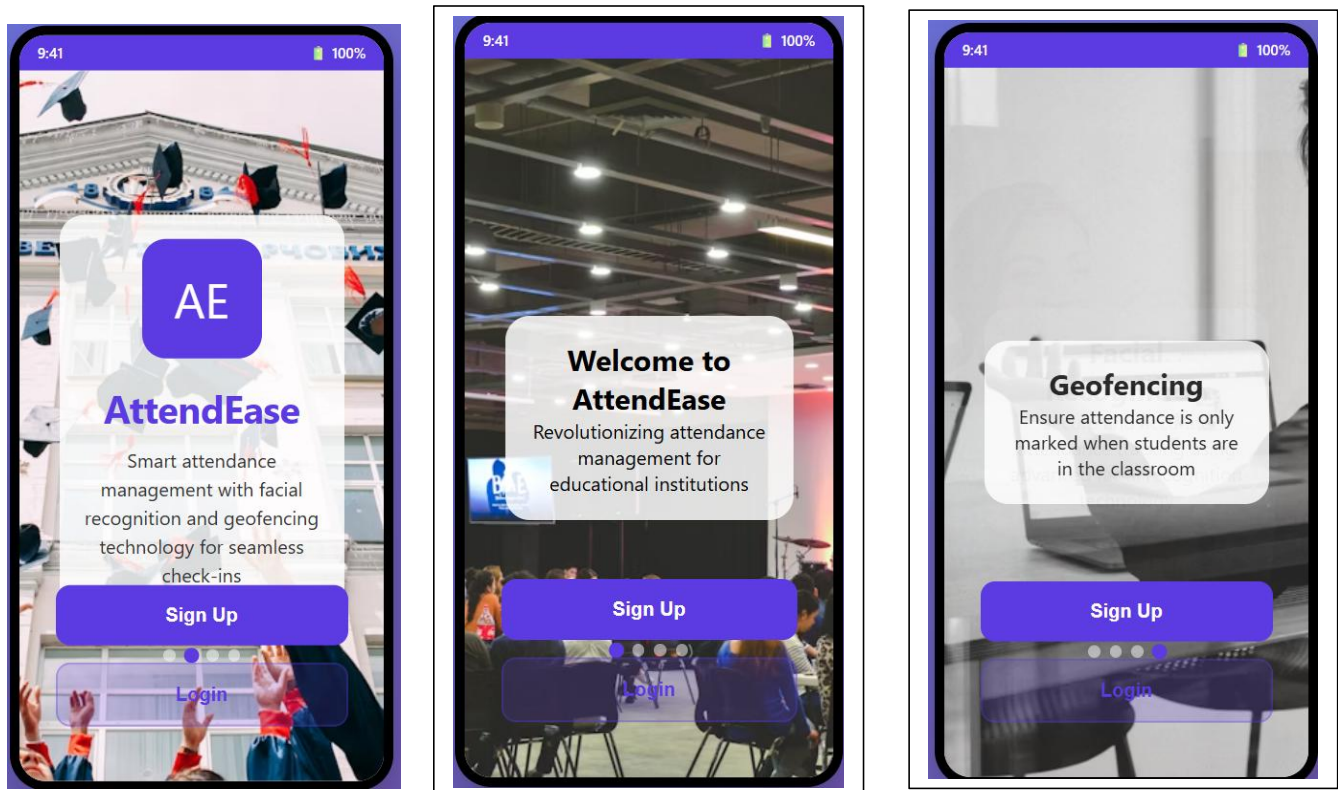


Figure 2: Figma design of the Welcome Screen with gradient background and action buttons.

B. Role Selection Screen Design

The Role Selection screen (role_selection.dart) is designed as a modal bottom sheet, presenting three role options (student, lecturer, admin) with corresponding emojis and descriptions. Each option is interactive, leading to the respective sign-up screens, ensuring a user-friendly role selection process.

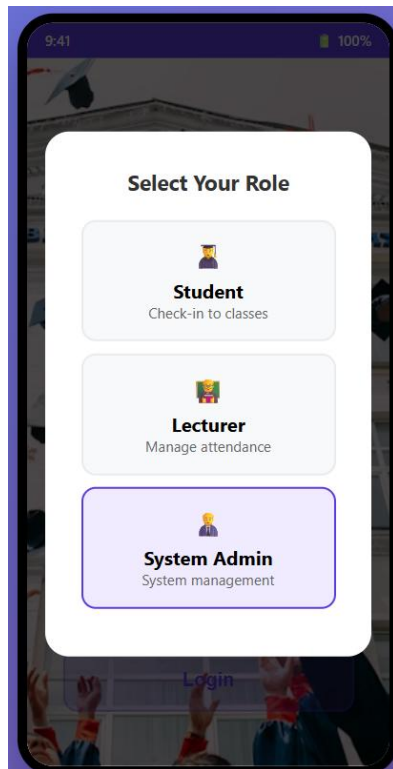


Figure 3: Figma design of the Role Selection Screen with role options.

C. Sign-Up and login Screens

Separate sign-up screens were designed for each user role (student, lecturer, admin), reflecting their unique needs. The student sign-up screen (`student_signup_screen.dart`) includes fields for matriculation number, department, level, and gender, along with a face registration section. The lecturer (`lecturer_signup_screen.dart`) and admin (`admin_signup_screen.dart`) sign-up screens include role-specific fields like lecturer ID and organization name, respectively.

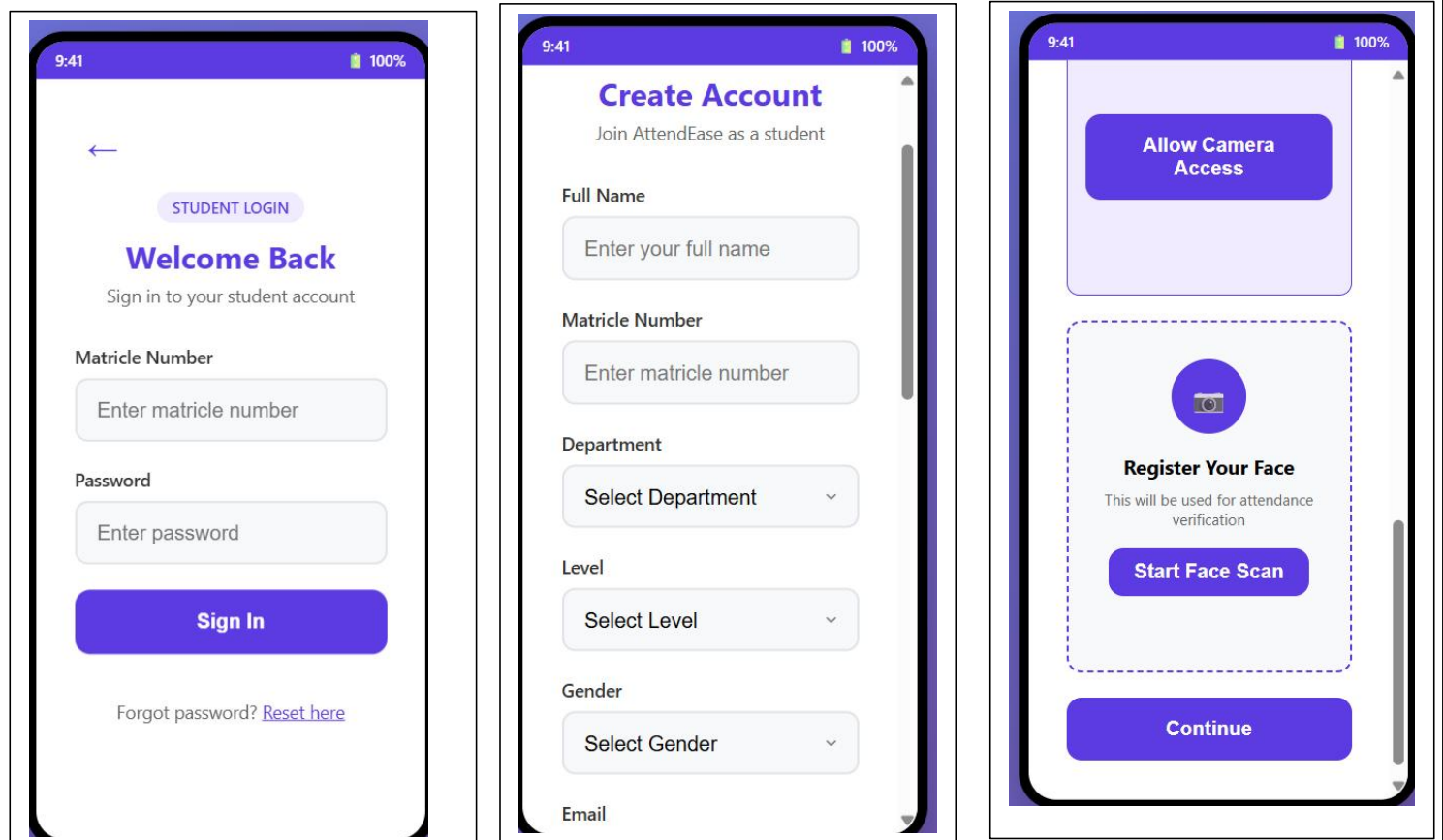


Figure 4: Figma designs of the Student login and signup

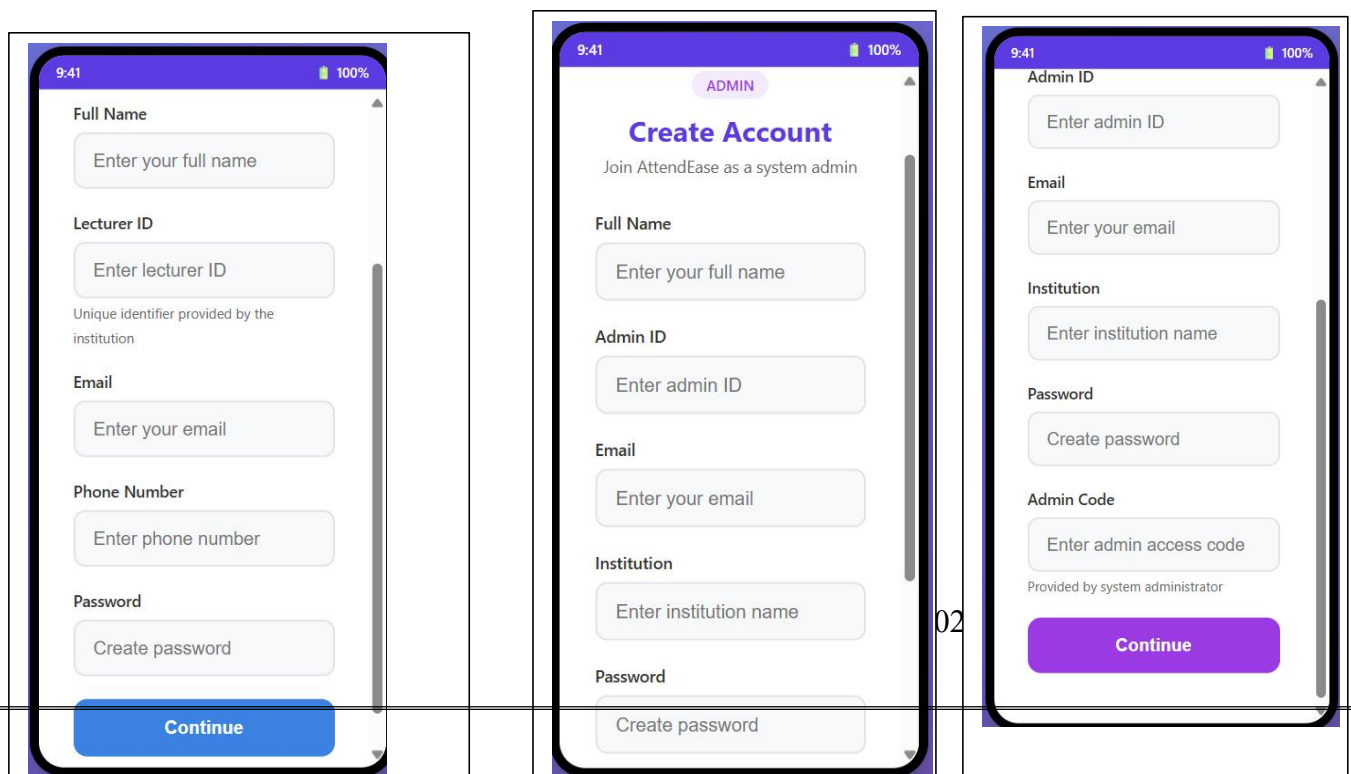


Figure 5: Lecturer and admin signup pages

Verification and Two-Factor Authentication (2FA) Screens

The verification screen (`verification_screen.dart`) and the new Two-Factor Authentication screen (`two_fa_screen.dart`) feature a clean layout with a 6-digit OTP input field, styled with role-specific primary colors (e.g., `AppConstants.primaryBlue`). The 2FA screen includes a gradient background and a shadow-boxed container for a modern aesthetic, with a status bar mockup (time and battery) and a dedicated 2FA section highlighted in green. The success screen (`success_screen.dart`) uses a Lottie animation to provide visual feedback upon successful account creation, followed by a navigation button to the login screen.

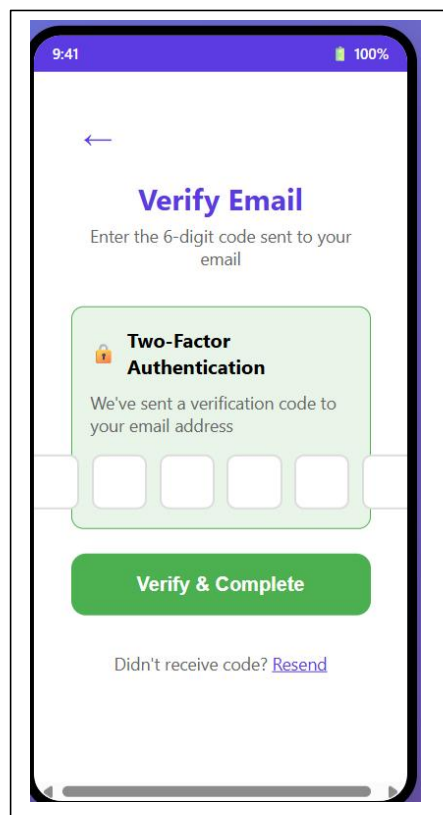


Figure 6: Figma designs of the Verification and Two-Factor Authentication Screens.

Success Screens

The success screen (success_screen.dart) uses a Lottie animation to provide visual feedback upon successful account creation, followed by a navigation button to the login screen.

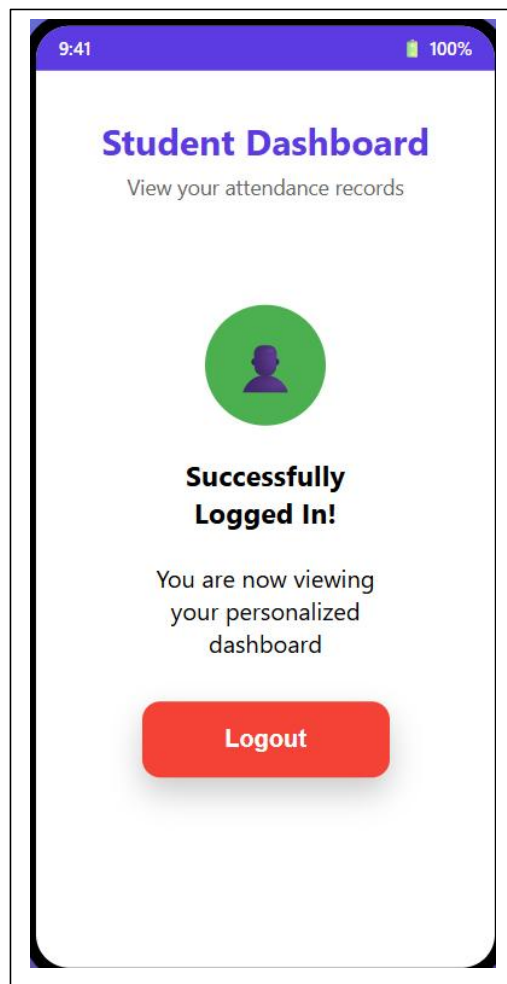


Figure 7: Figma designs of the Verification and Success Screens.

D. Dashboard Designs

i. Lecturer Dashboard Design

Dashboard Overview

Main Functionalities

The Lecturer Dashboard enables lecturers to:

- ❖ Start new attendance sessions for courses.
- ❖ Monitor live attendance in real time.
- ❖ View and manage their courses.
- ❖ Add new courses.
- ❖ Access their profile and logout.

Layout Summary

- ❖ **Header Bar:** Title, notifications with badge.
- ❖ **Quick Actions:** Grid of cards for key actions (Start Session, Live Attendance, etc.).
- ❖ **Active Courses:** List of courses with session status indicators.
- ❖ **Bottom Navigation:** Access to Home, Courses, Live, and Profile tabs.

The dashboard integrates geofencing and facial recognition by enabling session activation, which triggers student check-ins.

Design Justification Using Nielsen's Heuristics

Nielsen's Heuristics applied to the Lecturer Dashboard.

Heuristic	Flutter Implementation
Visibility of System Status	SnackBar, status indicators (Active/Inactive badges) for sessions.

Heuristic	Flutter Implementation
Match Between System and Real World	Terms like “Start Session”, “Live Attendance” align with lecturer workflows.
User Control and Freedom	Back buttons, Navigator.pop(), cancel options in dialogs.
Consistency and Standards	Uniform icons, button styles (CustomButton), color palette.
Error Prevention	Dropdown validation in session start dialog.
Recognition Rather Than Recall	Icon-labeled navigation, clear card titles.
Flexibility and Efficiency	Quick access via NavigationBar, state management with setState.
Aesthetic and Minimalist Design	Card layouts, white space, minimal text.
Help Users Recognize, Diagnose, and Recover from Errors	SnackBars (“Session started successfully”), retry/cancel in dialogs.
Help and Documentation	Inline helper text in dialogs (e.g., “Select a course”).

Key Design Principles Applied

Design principles applied to the Lecturer Dashboard.

Principle	Application
Hierarchy	GridView prioritizes actions (Start Session, Live Attendance).
Feedback	SnackBars, real-time status updates via setState.
Affordance	Tappable cards (InkWell) and icons indicate interactivity.
Consistency	Reusable _buildActionCard, standardized colors.
Mobile Responsiveness	MediaQuery, Expanded for layout adaptation.
Accessibility	Sufficient color contrast, readable fonts, semantic icons.

Detailed Components and Features

Header Bar

- Gradient background, white text.
- Notification icon with badge (e.g., “3” unread).
- Reinforces app branding.

Quick Actions (Cards)

Quick Action cards in the Lecturer Dashboard.

Icon	Title	Function
Icons.play_circle	Start Session	Opens dialog to select course.
Icons.people	Live Attendance	Navigates to real-time check-in screen.
Icons.book	My Courses	Lists lecturer’s courses.
Icons.add	Add Course	Modal to register new course.

Each card changes color based on active/inactive state.

Active Courses List

- Displays course code (e.g., CSC201), title, and status.
- Status is color-coded (green for active, grey for inactive).
- Clickable for deeper interaction.

Bottom Navigation

- Icons: Home, Courses, Live, Profile.
- Current tab highlighted.
- Persistent across app.

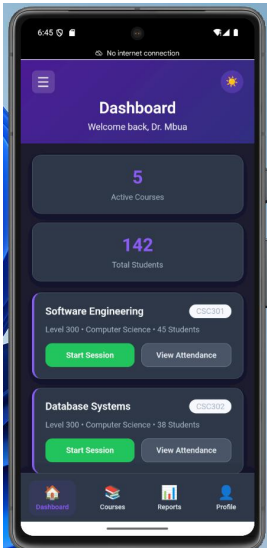
Modal Popups

- Used for: Start Session, Add Course.

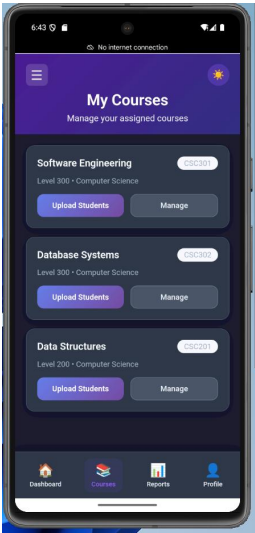
- Rounded corners, dropdowns, form validation.
- Closeable via button or outside click.

Strengths and Impact

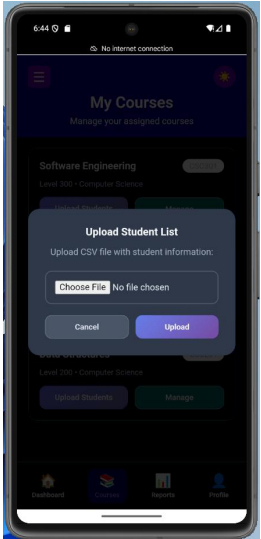
- **Task-Focused:** Key actions in 1–2 clicks.
- **Fast Feedback:** SnackBars and live updates.
- **Mobile-Optimized:** Clean layout for small screens.
- **Secure Controls:** Restricted to lecturer role.



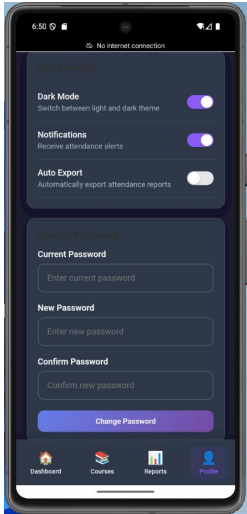
DASHBOARD



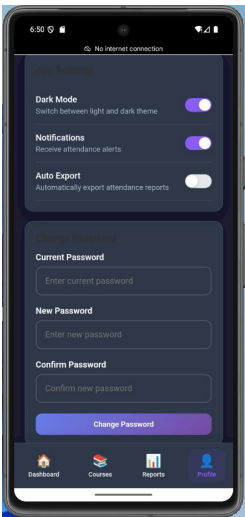
COURSE



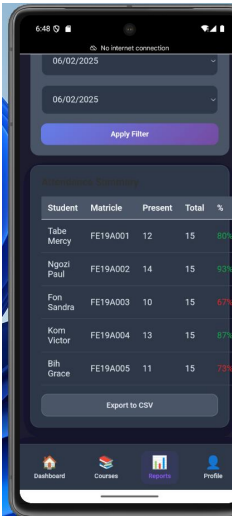
COURSE LIST UPLOAD



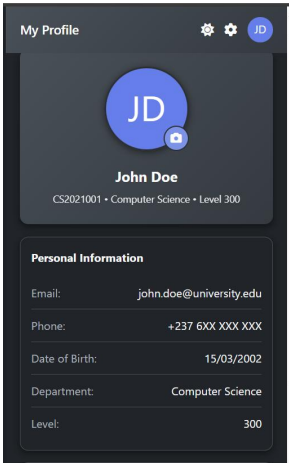
SETTINGS



SETTINGS



REPORT FORMAT



PROFILE

MIC YEAR

ii.Student Dashboard Design

Dashboard Overview

Main Functionalities

The Student Dashboard enables students to:

- Check in to active course sessions.
- View attendance records (percentage, present/absent counts).
- Access today's classes with session status.
- View notifications and profile.

Layout Summary

- **Header Bar:** Title, notifications with badge.
- **Welcome Card:** Personalized greeting with student details.
- **Quick Stats:** Attendance metrics.
- **Today's Classes:** List of classes with check-in option.
- **Bottom Navigation:** Access to Home, Courses, Attendance, Profile tabs.

The dashboard integrates geofencing and facial recognition for session check-ins.

Responsive Design

Uses SingleChildScrollView, Expanded, and MediaQuery for adaptability.

Design Justification Using Nielsen's Heuristics

Nielsen's Heuristics applied to the Student Dashboard.

Heuristic	Flutter Implementation
Visibility of System Status	Class status badges (Present, Pending, Absent).
Match Between System and Real World	Terms like "Check In", "Today's Classes" align

Heuristic	Flutter Implementation
	with student workflows.
User Control and Freedom	Back buttons, Navigator.pop().
Consistency and Standards	Uniform icons, CustomButton, color palette.
Error Prevention	Check-in button disabled if session inactive.
Recognition Rather Than Recall	Icon-labeled navigation, clear class details.
Flexibility and Efficiency	Quick check-in via CustomButton.
Aesthetic and Minimalist Design	Clean card layouts, minimal text.
Help Users Recognize, Diagnose, and Recover from Errors	SnackBar for check-in success/failure.
Help and Documentation	Inline status text (e.g., “Session not active”).

Key Design Principles Applied

Design principles applied to the Student Dashboard.

Principle	Application
Hierarchy	Welcome card and stats prioritized at the top.
Feedback	Real-time check-in status updates.
Affordance	Tappable CustomButton for check-in.
Consistency	Reusable _buildClassCard, standardized colors.
Mobile Responsiveness	MediaQuery, Expanded for layout.
Accessibility	High contrast, readable fonts, semantic icons.

Detailed Components and Features

Header Bar

- Gradient background, white text.
- Notification icon with badge (e.g., “2” unread).

- Theme toggle icon.

Welcome Card

- Gradient background, displays “Hello, [Student Name]!”.
- Shows department and level.

Quick Stats

- Displays attendance percentage, present, and absent counts.
- Color-coded for quick understanding.

Today’s Classes List

- Shows class title, lecturer, time, and status.
- Status color-coded (green for present, yellow for pending, red for absent).
- CustomButton for check-in if session is active.

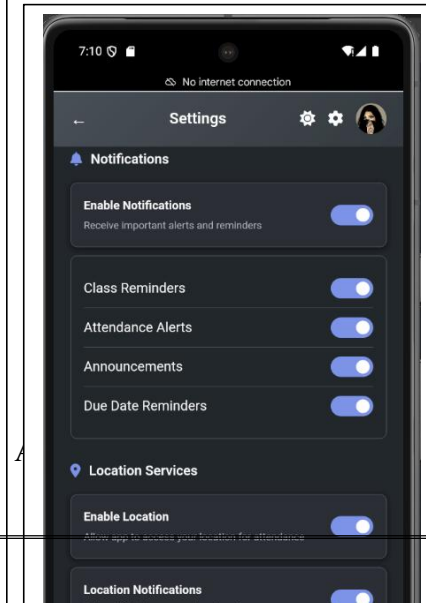
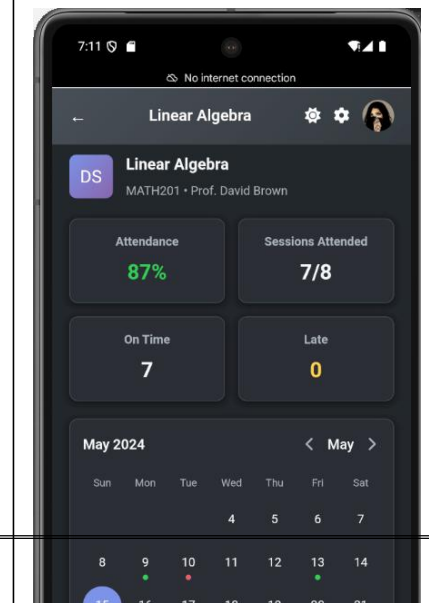
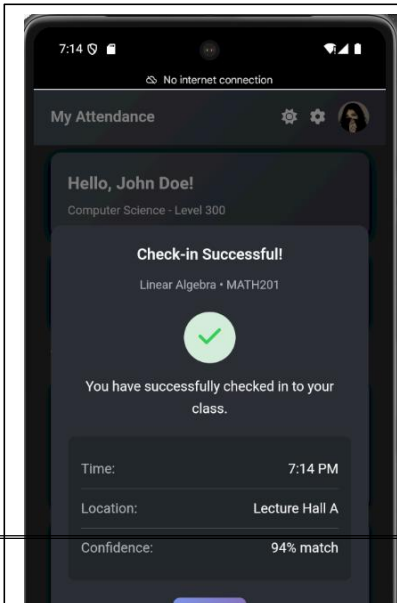
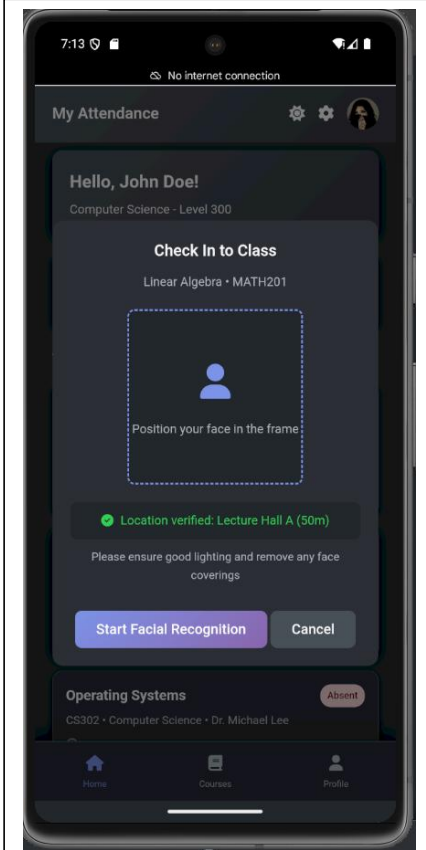
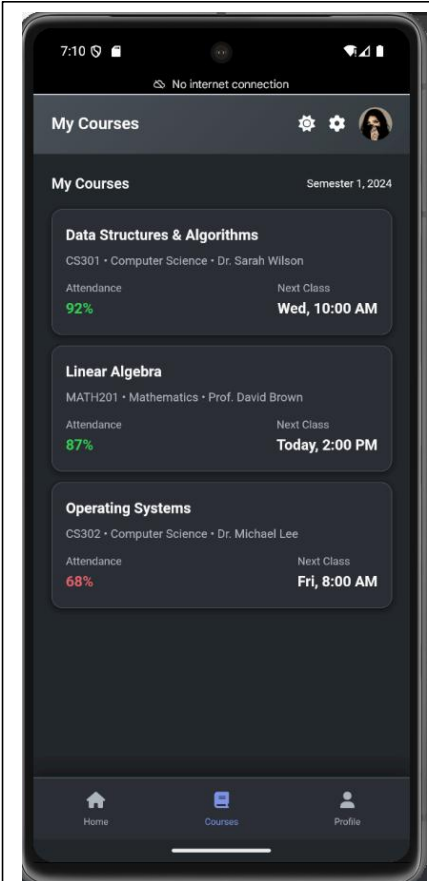
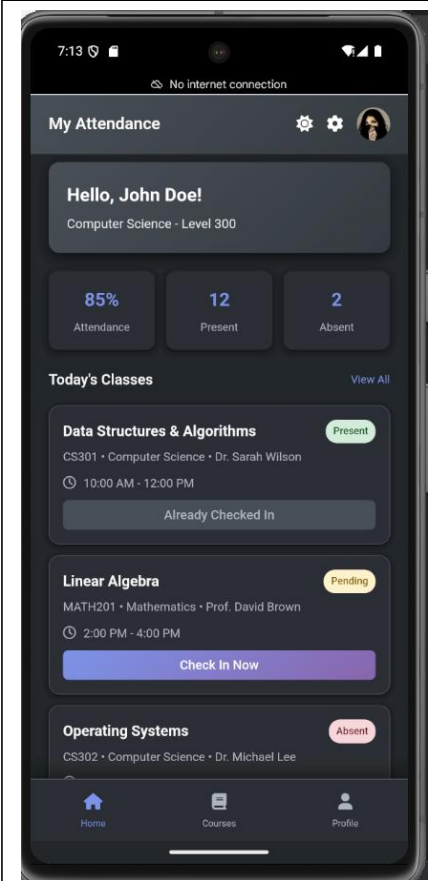
Bottom Navigation

- Icons: Home, Courses, Attendance, Profile.
- Current tab highlighted.
- Persistent across app.

Strengths and Impact

- **Task-Focused:** Quick check-in and attendance overview.
- **Fast Feedback:** Immediate check-in status updates.
- **Mobile-Optimized:** Clean layout for small screens.
- **Secure Controls:** Check-in restricted to active sessions.

Screenshots



iii.Admin Dashboard Design and Implementation

Dashboard Overview

Main Functionalities

The Admin Dashboard enables administrators to:

- View system stats (total students, lecturers, active sessions).
- Manage students, lecturers, and courses.
- Configure geofenced areas.
- Generate attendance reports.
- Access settings and profile.

Layout Summary

- **Header Bar:** Title, theme toggle, profile icon.
- **Quick Stats:** Overview of system metrics.
- **Quick Actions:** Grid of cards for management tasks.
- **Bottom Navigation:** Access to Dashboard, Students, Lecturers, Reports, Settings tabs.

The dashboard supports system-wide management with geofencing configuration.

Responsive Design

Uses SingleChildScrollView, Expanded, and MediaQuery for adaptability.

Design Justification Using Nielsen's Heuristics

Nielsen's Heuristics applied to the Admin Dashboard.

Heuristic	Flutter Implementation
Visibility of System Status	Stat cards with real-time metrics.
Match Between System and Real World	Terms like "Manage Students", "Reports" align with admin tasks.
User Control and Freedom	Back buttons, Navigator.pop().
Consistency and Standards	Uniform icons, card styles, color palette.
Error Prevention	Action cards disabled if prerequisites unmet.
Recognition Rather Than Recall	Icon-labeled navigation, clear card titles.
Flexibility and Efficiency	Quick access via NavigationBar.
Aesthetic and Minimalist Design	Clean card layouts, white space.
Help Users Recognize, Diagnose, and Recover from Errors	SnackBar for action feedback.
Help and Documentation	Inline labels for clarity (e.g., "Total Students").

Key Design Principles Applied

Design principles applied to the Admin Dashboard.

Principle	Application
Hierarchy	Stats and actions prioritized at the top.
Feedback	Real-time metric updates via setState.
Affordance	Tappable InkWell cards for actions.
Consistency	Reusable _buildActionCard, standardized colors.
Mobile Responsiveness	MediaQuery, Expanded for layout.
Accessibility	High contrast, readable fonts, semantic icons.

Detailed Components and Features

Header Bar

- Gradient background, white text.
- Theme toggle and profile icons.

Quick Stats

- Displays total students, lecturers, active sessions.
- Color-coded for quick understanding.

Quick Actions (Cards)

Quick Action cards in the Admin Dashboard.

Icon	Title	Function
Icons.people	Manage Students	Navigates to student management tab.
Icons.school	Manage Lecturers	Navigates to lecturer management tab.
Icons.book	Manage Courses	Opens course management screen.
Icons.location_on	Geofence Areas	Configures geofenced areas.
Icons.bar_chart	Reports	Navigates to reports tab.
Icons.settings	Settings	Navigates to settings tab.

Bottom Navigation

- Icons: Dashboard, Students, Lecturers, Reports, Settings.
- Current tab highlighted.
- Persistent across app.

Strengths and Impact

- **Task-Focused:** Quick system management.
- **Fast Feedback:** Real-time stats updates.

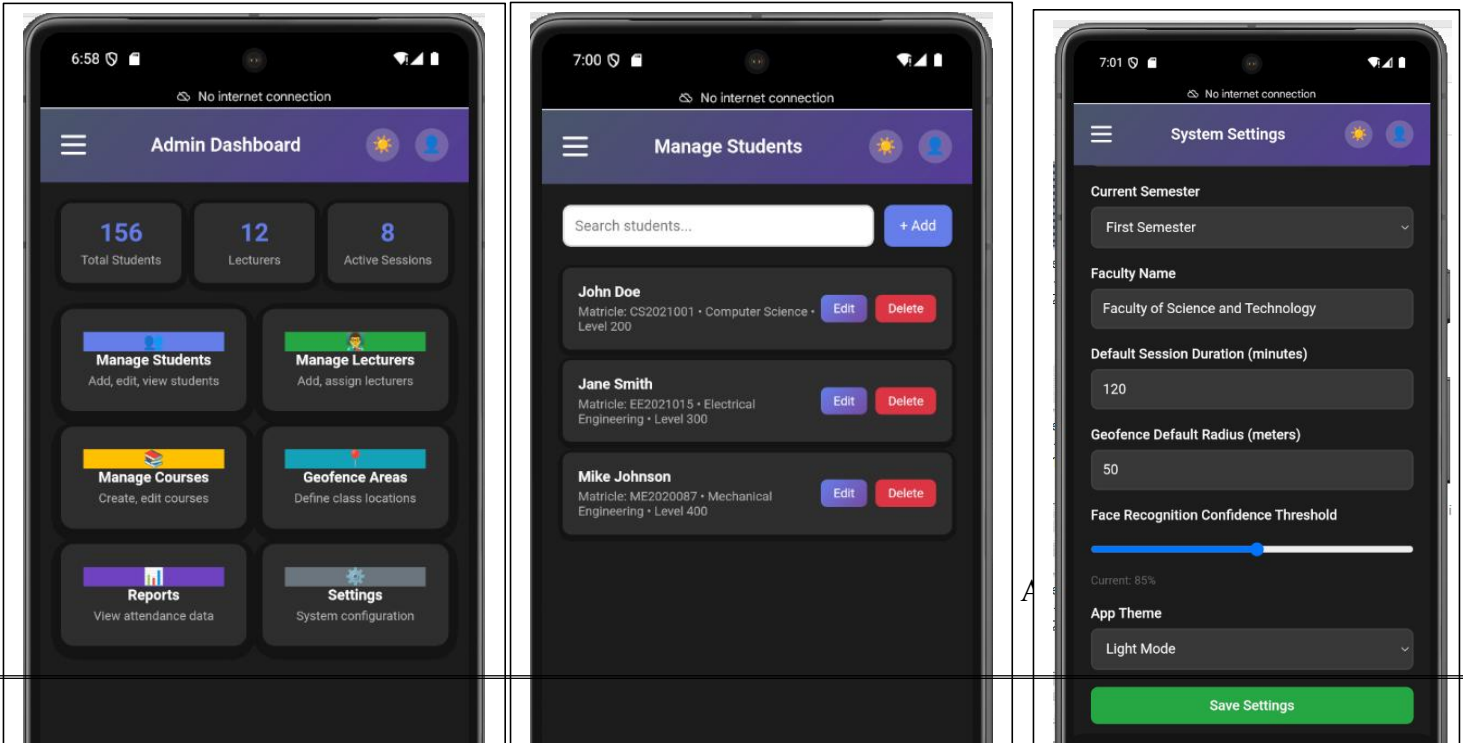
- **Mobile-Optimized:** Clean layout for small screens.
- **Secure Controls:** Restricted to admin role.

Challenges Faced

Challenges faced and their resolutions.

Challenge	Resolution
Multi-screen state management	Used setState with IndexedStack, explored provider for larger state sharing.
Real-time data simulation	Used mock data and Timer.periodic to simulate live attendance updates.
Responsive design across devices	Implemented MediaQuery and SingleChildScrollView for layout adaptability.
Geofencing integration	Configured geolocator permissions in AndroidManifest.xml, used mock locations for testing.
Facial recognition simulation	Simulated with placeholder UI, as full integration requires backend API.

Screenshots



Frontend Implementation

Technology Stack

The frontend of AttendEase was implemented using Flutter, a cross-platform framework by Google that allows for the development of iOS and Android apps from a single codebase. Dart, Flutter's programming language, was used to write the app's logic. The app was tested on an Android emulator, as shown in the Visual Studio Code (VSCode) screenshot, ensuring compatibility with Android devices.

Folder Structure

The project follows a structured organization in VSCode, as depicted in the provided screenshot:

- **android/:** Contains Android-specific configurations, including AndroidManifest.xml for app permissions and metadata.
- **ios/:** Placeholder for iOS-specific configurations (not used in this project due to Android focus).
- **lib/:** Main Dart code directory.
 - **core/:** Includes constants (colors.dart), themes (app_theme.dart), utilities, and reusable widgets.
 - **features/:** Feature-specific code.
 - **auth/:** Authentication screens (login_screen.dart, signup_screens.dart, verification_screen.dart, success_screen.dart).
 - **dashboard/:** Dashboard screens for each role.

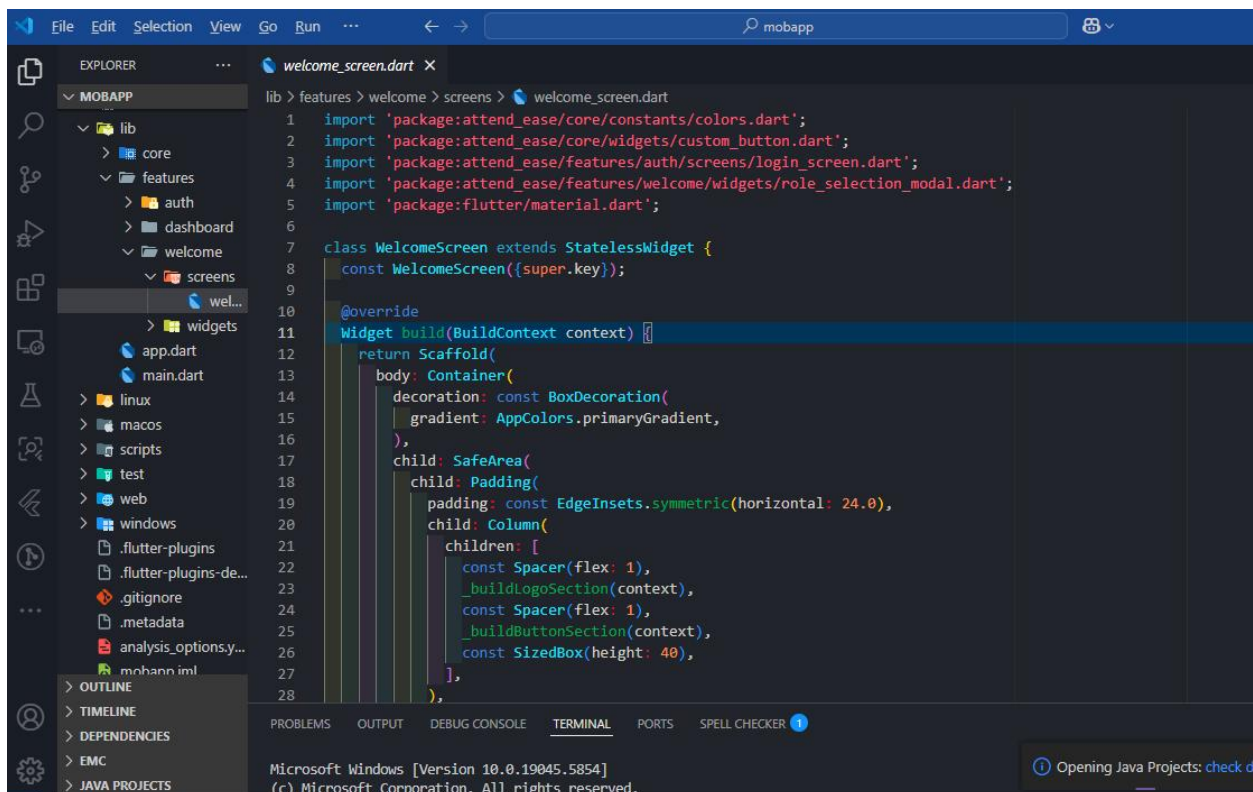
- welcome/: Welcome screen and role selection modal.
- **main.dart**: Entry point of the app.

File Usage and Implementation Details

Below is a detailed breakdown of the key files in the lib/ directory and their roles in the implementation.

Welcome Screen (welcome_screen.dart)

The welcome screen serves as the app's entry point, featuring a gradient background, logo, and two buttons for sign-up and login. It uses the RoleSelectionModal widget to allow users to choose their role before signing up.



```

1  import 'package:attend_ease/core/constants/colors.dart';
2  import 'package:attend_ease/core/widgets/custom_button.dart';
3  import 'package:attend_ease/features/auth/screens/login_screen.dart';
4  import 'package:attend_ease/features/welcome/widgets/role_selection_modal.dart';
5  import 'package:flutter/material.dart';
6
7  class WelcomeScreen extends StatelessWidget {
8    const WelcomeScreen({super.key});
9
10   @override
11   Widget build(BuildContext context) {
12     return Scaffold(
13       body: Container(
14         decoration: const BoxDecoration(
15           gradient: AppColors.primaryGradient,
16         ),
17       child: SafeArea(
18         child: Padding(
19           padding: const EdgeInsets.symmetric(horizontal: 24.0),
20           child: Column(
21             children: [
22               const Spacer(flex: 1),
23               _buildLogoSection(context),
24               const Spacer(flex: 1),
25               _buildButtonSection(context),
26               const SizedBox(height: 40),
27             ],
28           ),
29         ),
30       ),
31     );
32   }
33 }

```

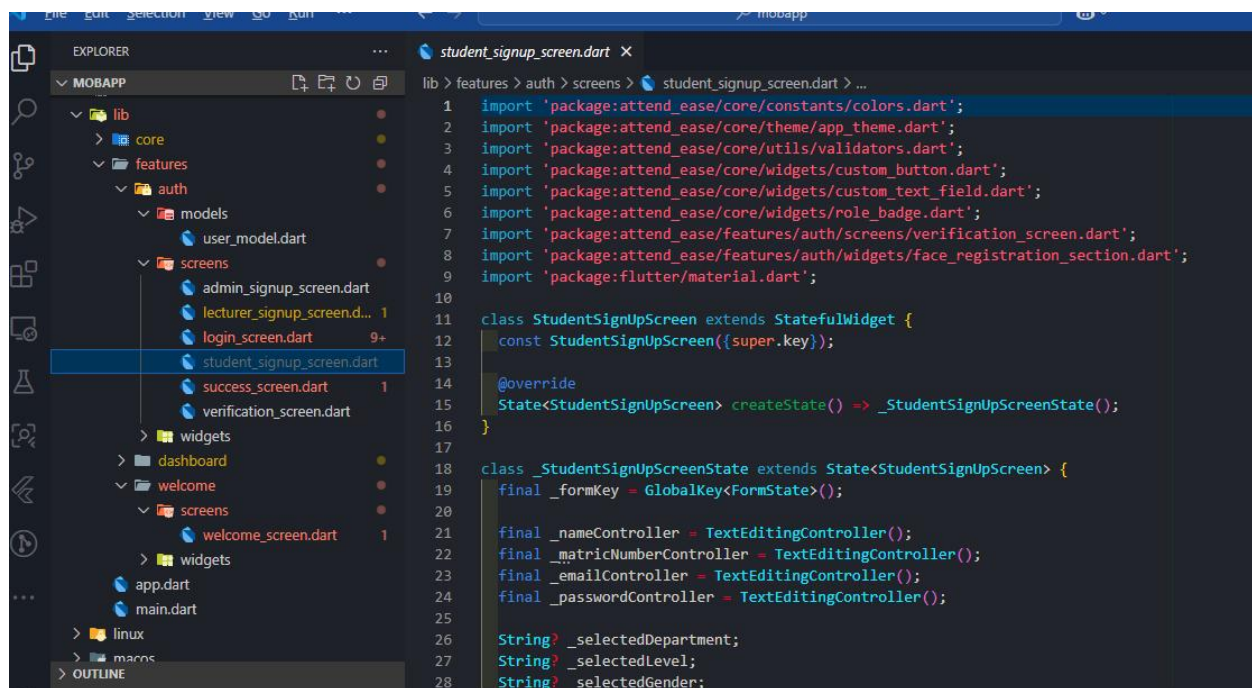
Figure 6: Code snippet of welcome_screen.dart showing the gradient background and button implementation.

Sign-Up Screens

Each role has a dedicated sign-up screen:

- `student_signup_screen.dart`: Includes dropdown fields for department, level, and gender, and integrates the `FaceRegistrationSection` for facial recognition setup.
- `lecturer_signup_screen.dart`: Features a lecturer ID field with a helper text for clarity.
- `admin_signup_screen.dart`: Includes an organization field to capture the admin's institution.

All sign-up screens validate user input using the `Validators` utility and navigate to the verification screen upon successful validation.



```
lib > features > auth > screens > student_signup_screen.dart > ...
1  import 'package:attend_ease/core/constants/colors.dart';
2  import 'package:attend_ease/core/theme/app_theme.dart';
3  import 'package:attend_ease/core/utils/validators.dart';
4  import 'package:attend_ease/core/widgets/custom_button.dart';
5  import 'package:attend_ease/core/widgets/custom_text_field.dart';
6  import 'package:attend_ease/core/widgets/role_badge.dart';
7  import 'package:attend_ease/features/auth/screens/verification_screen.dart';
8  import 'package:attend_ease/features/auth/widgets/face_registration_section.dart';
9  import 'package:flutter/material.dart';
10
11 class StudentSignUpScreen extends StatefulWidget {
12   const StudentSignUpScreen({super.key});
13
14   @override
15   State<StudentSignUpScreen> createState() => _StudentSignUpScreenState();
16 }
17
18 class _StudentSignUpScreenState extends State<StudentSignUpScreen> {
19   final _formKey = GlobalKey<FormState>();
20
21   final _nameController = TextEditingController();
22   final _matricNumberController = TextEditingController();
23   final _emailController = TextEditingController();
24   final _passwordController = TextEditingController();
25
26   String? _selectedDepartment;
27   String? _selectedLevel;
28   String? selectedGender;
```

Figure 7: Code snippet of `student_signup_screen.dart`

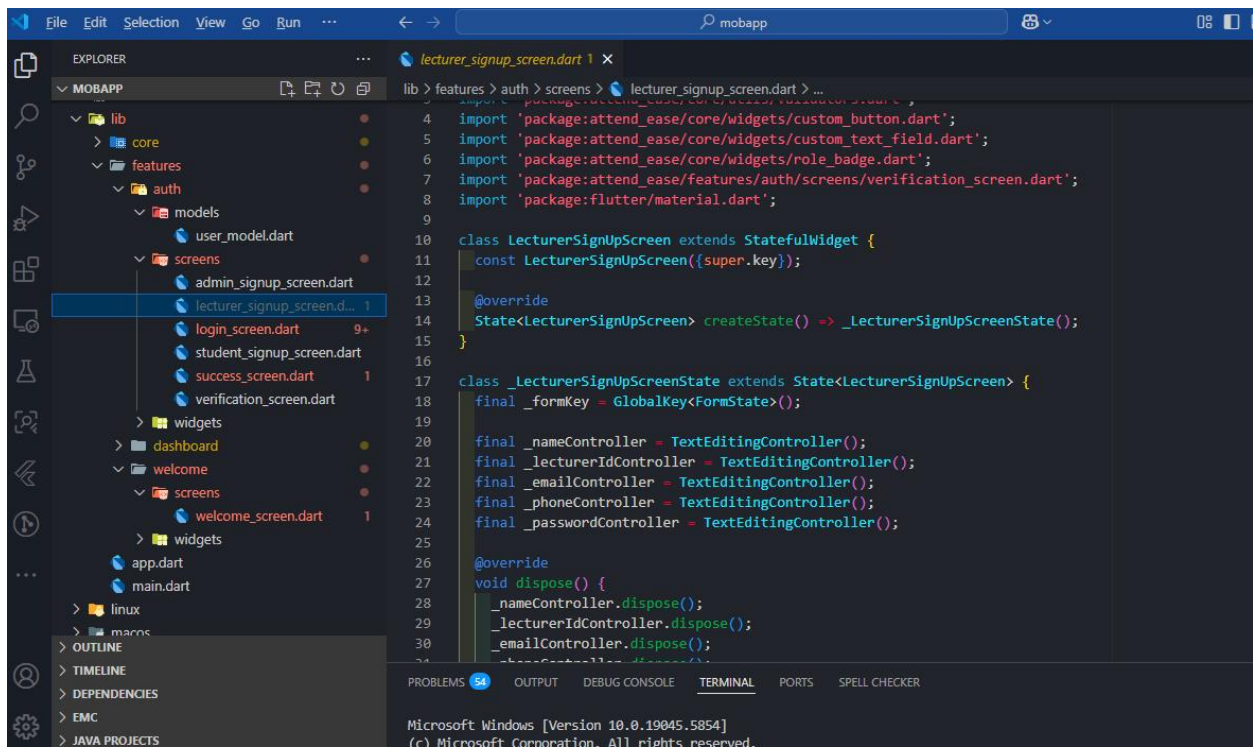


Figure 8: Lecturer_signup_screen

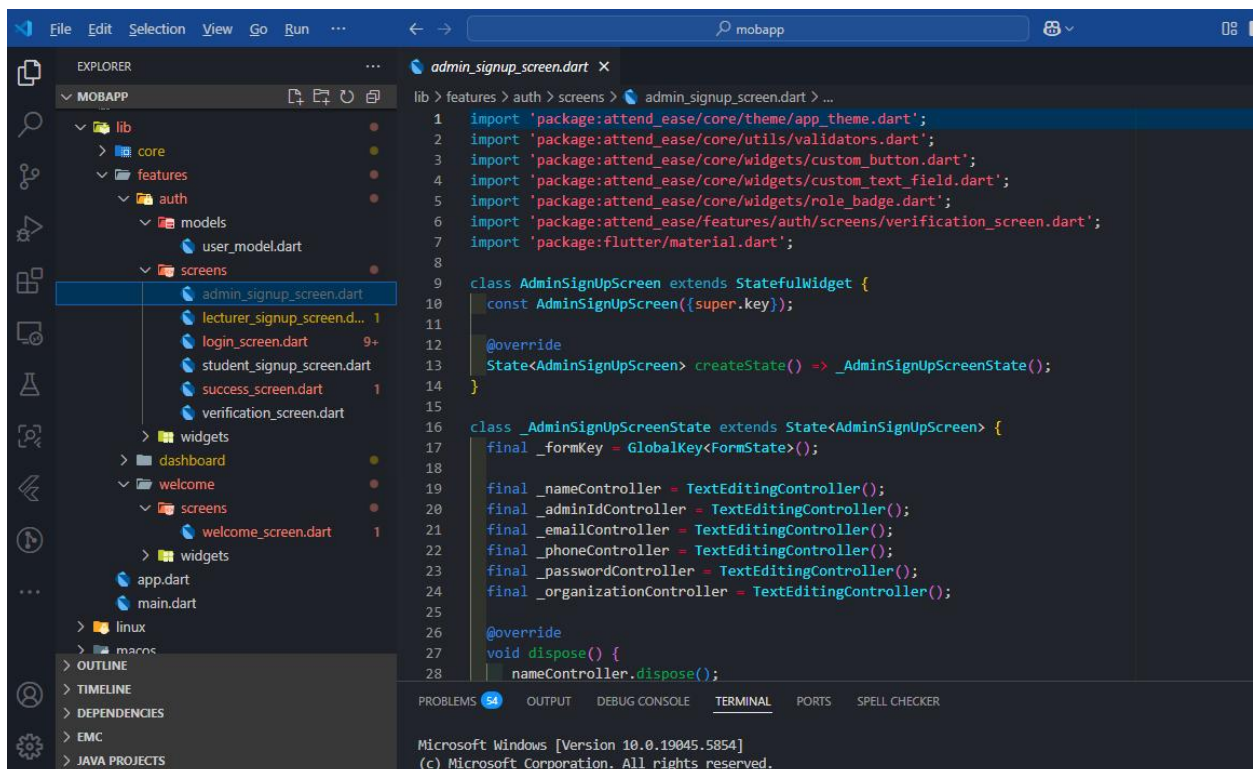


Figure 9:Admin_signup_screen.dart.

Verification and Success Screens

- **verification_screen.dart:** Implements a 6-digit OTP input using the PinCodeTextField widget from the pin_code_fields package. It supports auto-submission upon OTP completion and includes a resend option.
- **success_screen.dart:** Displays a Lottie animation for visual feedback and navigates to the login screen.

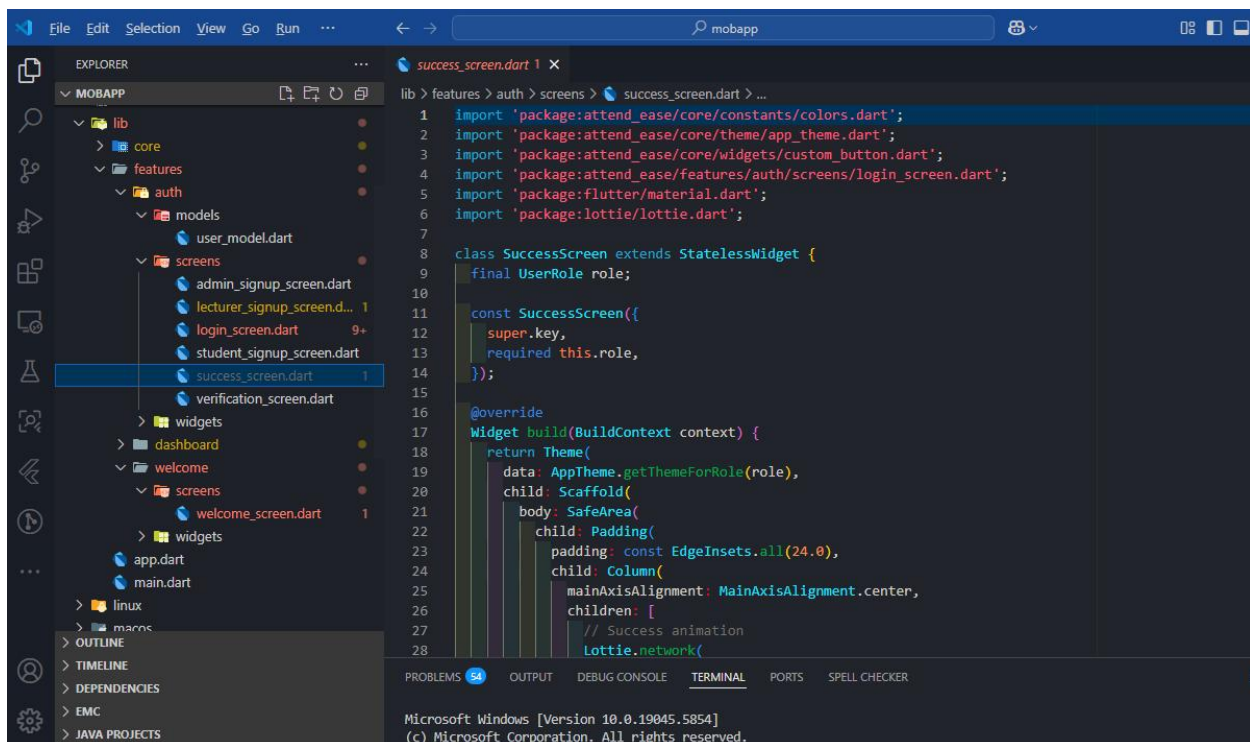


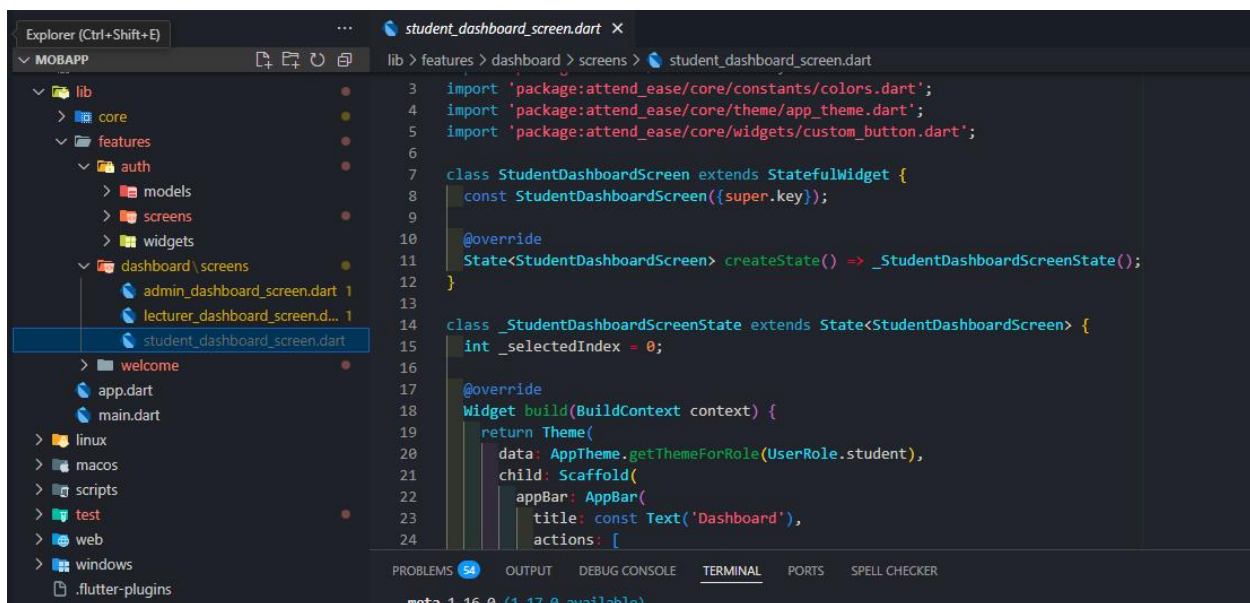
Figure 10: Code snippets of verification_screen.dart and success_screen.dart.

Dashboards and Role-Specific Functionalities

Upon successful login, users are redirected to their respective dashboards based on their role, as implemented in login_screen.dart. Each dashboard offers tailored functionalities:

- **Student Dashboard (student_dashboard_screen.dart):** When students log in successfully, they are redirected to the student dashboard. Here, they can:
 - Check in for a particular course whose session has been activated by the lecturer, using geofencing and facial recognition verification.
 - View their attendance record, including attendance percentage, present/absent counts, and a list of today's classes with statuses (e.g., pending, present, absent).
 - Access notifications via the notification bell icon, which displays unread alerts (e.g., session reminders).

The dashboard uses an IndexedStack for tab navigation between Home, Courses, Attendance, and Profile tabs.



```

lib > features > dashboard > screens > student_dashboard_screen.dart
3  import 'package:attend_ease/core/constants/colors.dart';
4  import 'package:attend_ease/core/theme/app_theme.dart';
5  import 'package:attend_ease/core/widgets/custom_button.dart';
6
7  class StudentDashboardScreen extends StatefulWidget {
8    const StudentDashboardScreen({super.key});
9
10   @override
11   State<StudentDashboardScreen> createState() => _StudentDashboardScreenState();
12 }
13
14 class _StudentDashboardScreenState extends State<StudentDashboardScreen> {
15   int _selectedIndex = 0;
16
17   @override
18   Widget build(BuildContext context) {
19     return Theme(
20       data: AppTheme.getThemeForRole(UserRole.student),
21       child: Scaffold(
22         appBar: AppBar(
23           title: const Text('Dashboard'),
24           actions: [

```

Figure 11: Code snippets of student_dashboard_screen.dart

- **Lecturer Dashboard (lecturer_dashboard_screen.dart):** Upon successful login, lecturers are redirected to the lecturer dashboard. Here, they can:
 - Start a new session for a selected course using a dialog interface, enabling attendance tracking within a geofenced area.

- View live attendance for active sessions, displaying real-time check-in data from students.
- Manage courses by adding new ones or viewing a list of active/inactive courses.
- Receive notifications via the notification bell icon for session-related alerts.

The dashboard uses an IndexedStack for navigation between Home, Courses, Live, and Profile tabs.

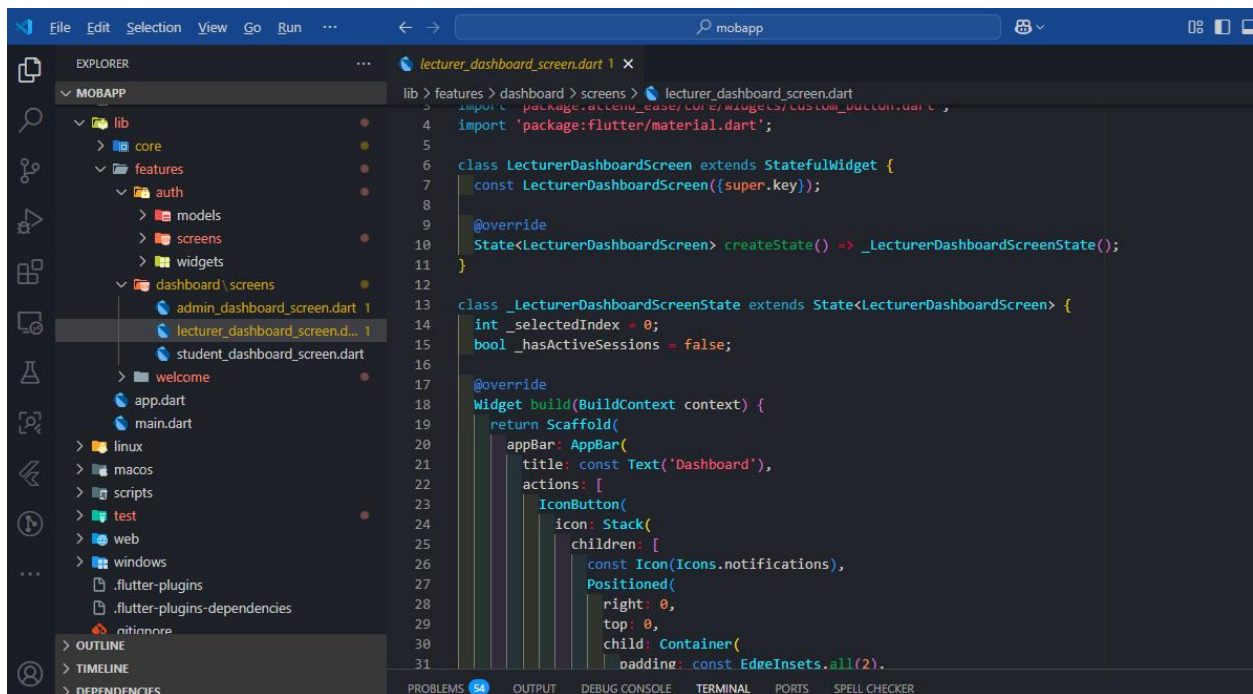


Figure 12:Lecturer_dashboard_screen.dart

- **Admin Dashboard (admin_dashboard_screen.dart):** After successful login, administrators are redirected to the admin dashboard. Here, they can:
 - View quick stats (total students, lecturers, active sessions) to monitor system usage.

- Manage students and lecturers through dedicated tabs, including adding or editing user details.
- Manage courses and geofenced areas, ensuring proper configuration for attendance tracking.
- Generate reports for attendance data and system performance.
- Access settings for theme toggling and profile management.

The dashboard uses an IndexedStack for navigation between Dashboard, Students, Lecturers, Reports, and Settings tabs.

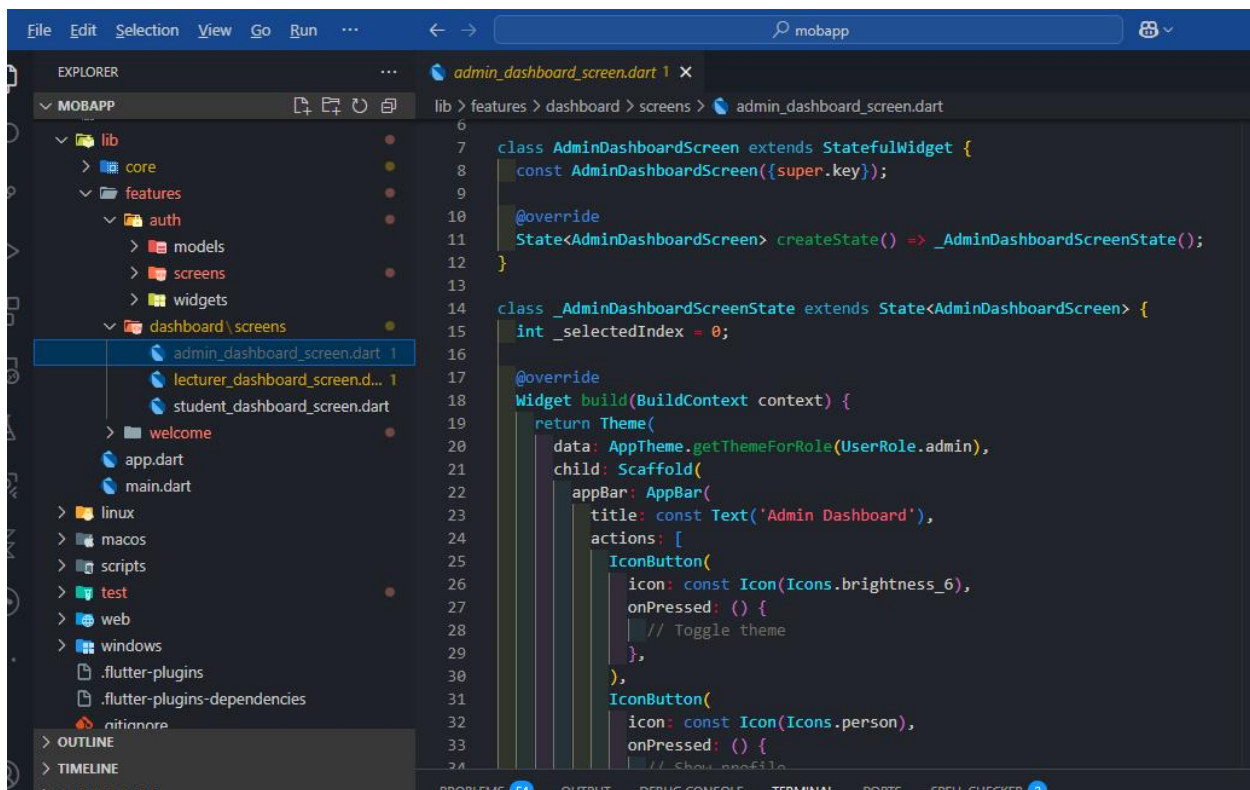


Figure 13: Admin_dashboard_screen.dart.

Login Screen (login_screen.dart)

The login screen supports role-based navigation, directing users to their respective dashboards (student, lecturer, or admin) upon successful login.

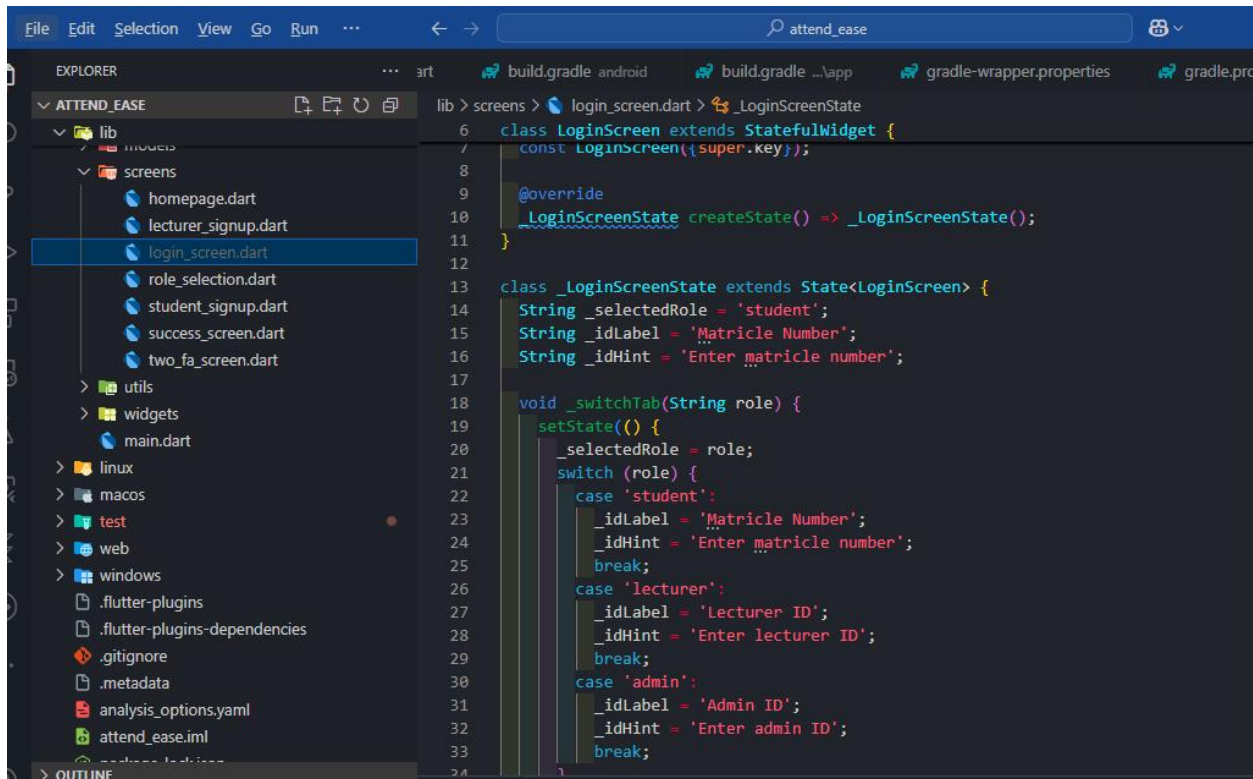


Figure 14: Code snippet of login_screen.dart showing role-based navigation.

Main and App Dart Files

- main.dart: Serves as the entry point of the application. It initializes the Flutter app and runs the root widget defined in app.dart. Typically, it sets up the app's main method and calls runApp() with the app widget.
- app.dart: Defines the root widget of the application, often configuring the MaterialApp or CupertinoApp with themes, routes, and initial screens (e.g., the welcome screen). It integrates state management (via provider) and navigation logic.

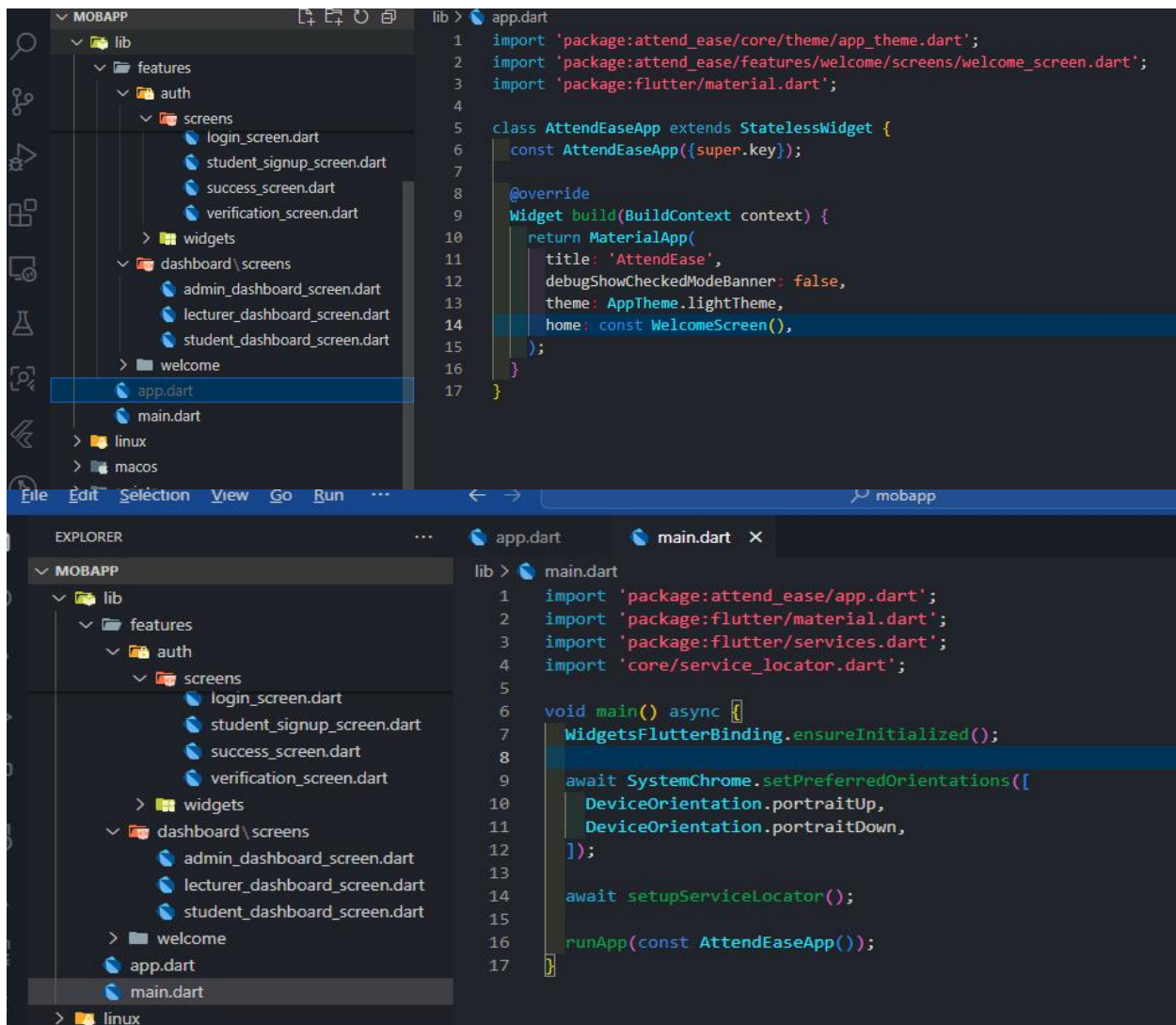


Figure 14: Code snippets of main.dart and app.dart showing app initialization and configuration.

Pubspec.yaml Configuration

The pubspec.yaml file is the project configuration file for Flutter applications. It defines the app's metadata (name: attend_ease, description, version: 1.0.0), SDK environment ($\geq 3.7.2$ $< 4.0.0$), and dependencies. Key dependencies include:

- flutter: Core SDK for building the app.
- cupertino_icons: Provides iOS-style icons.

- `google_fonts`: Enables custom font usage.
- `provider`: State management solution.
- `geolocator`: Handles geofencing and location services.
- `google_ml_kit`: Supports face detection (requires backend API for full recognition).

The `dev_dependencies` section includes `flutter_test` for unit testing and `flutter_lints` for code quality. The `uses-material-design: true` flag enables Material Design widgets.

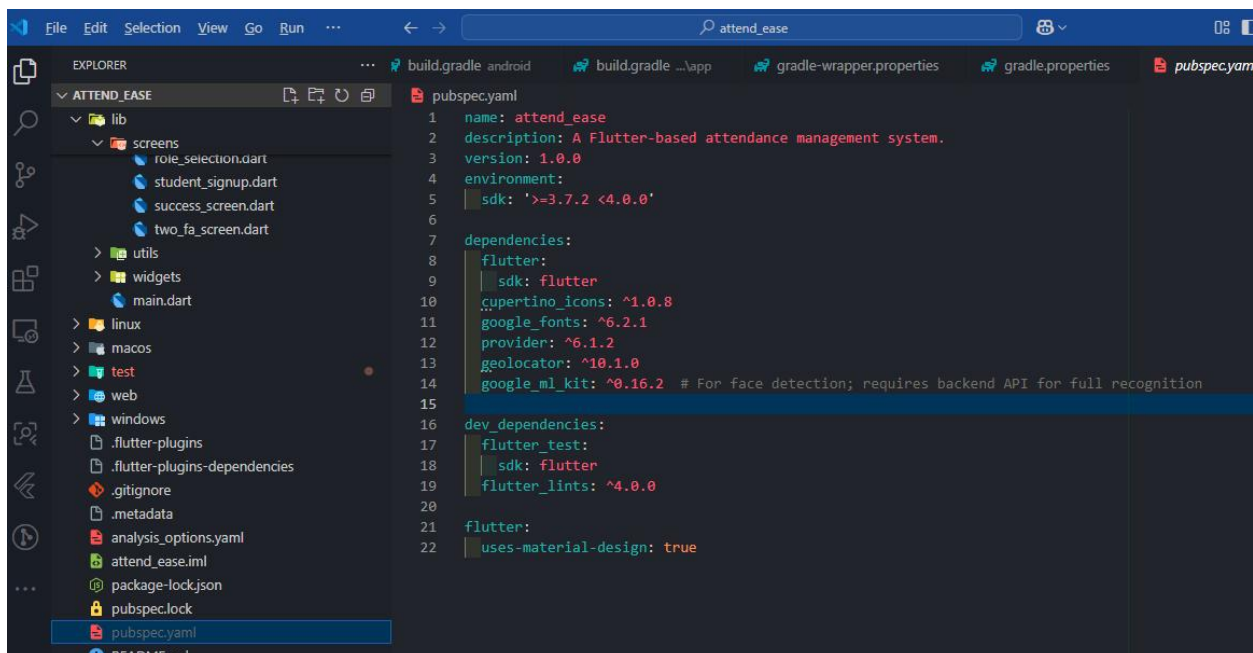
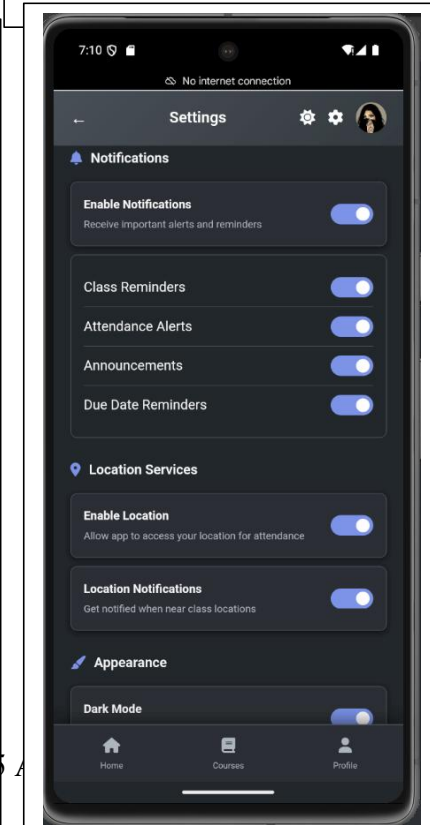
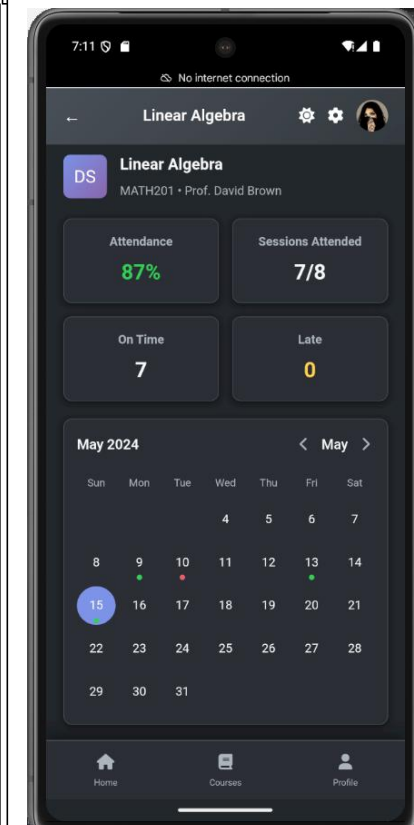
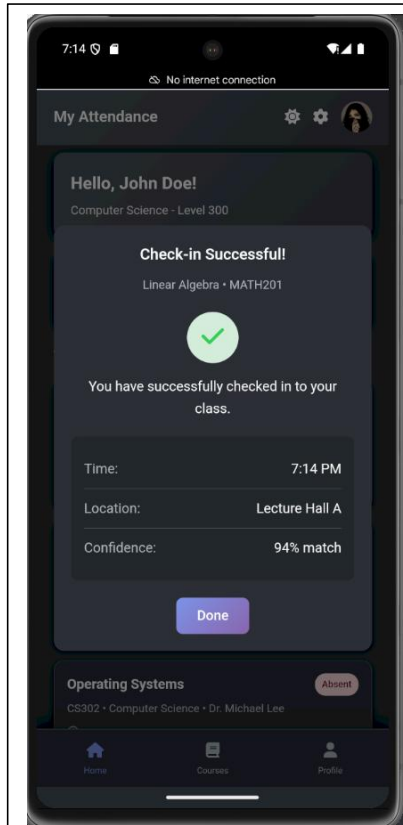
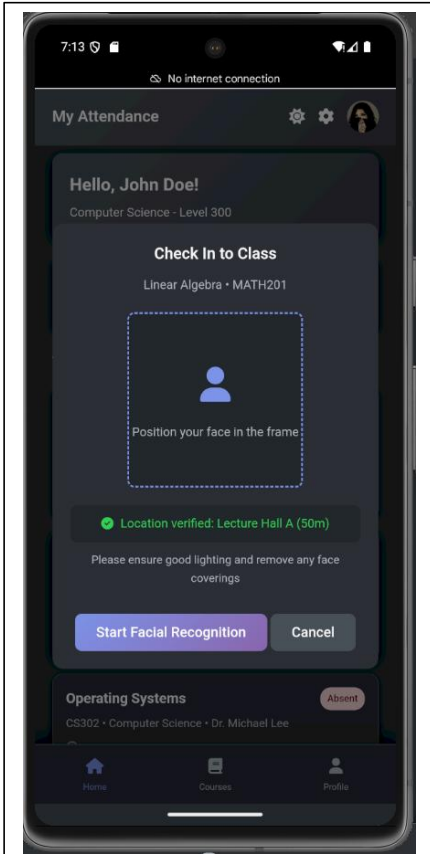
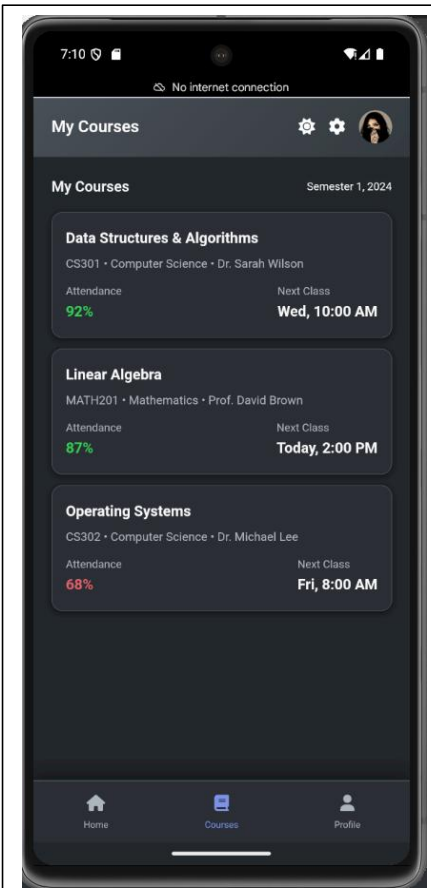
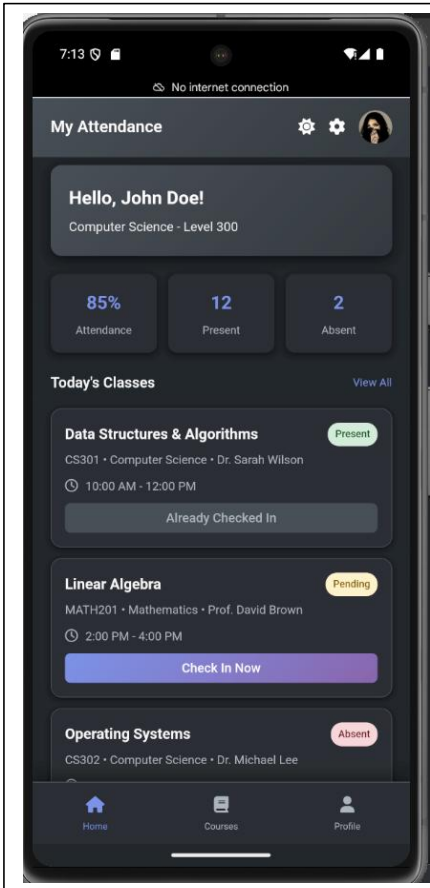


Figure 15: Content of `pubspec.yaml` showing dependencies and configuration.

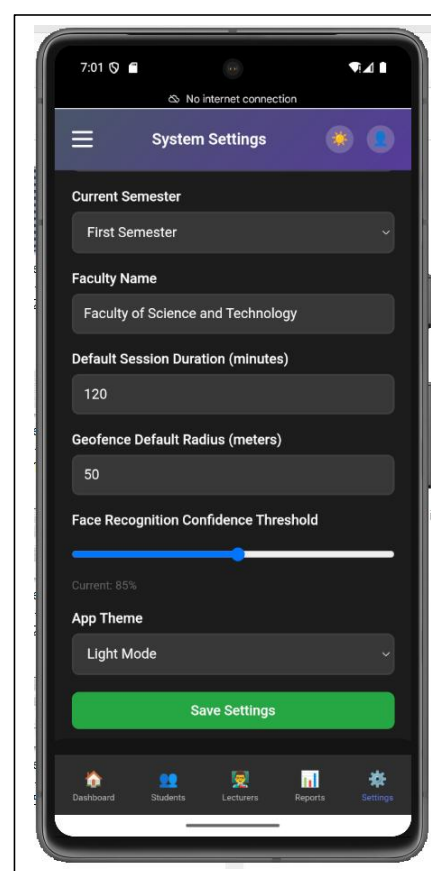
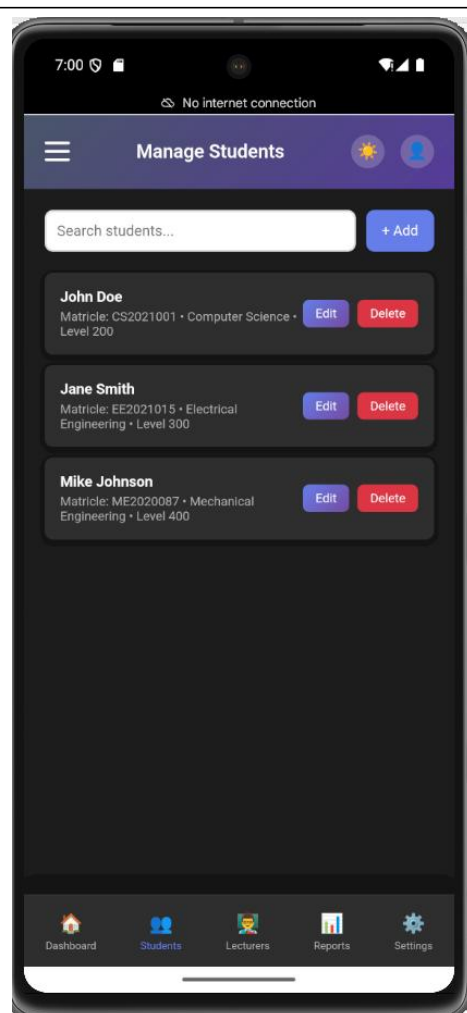
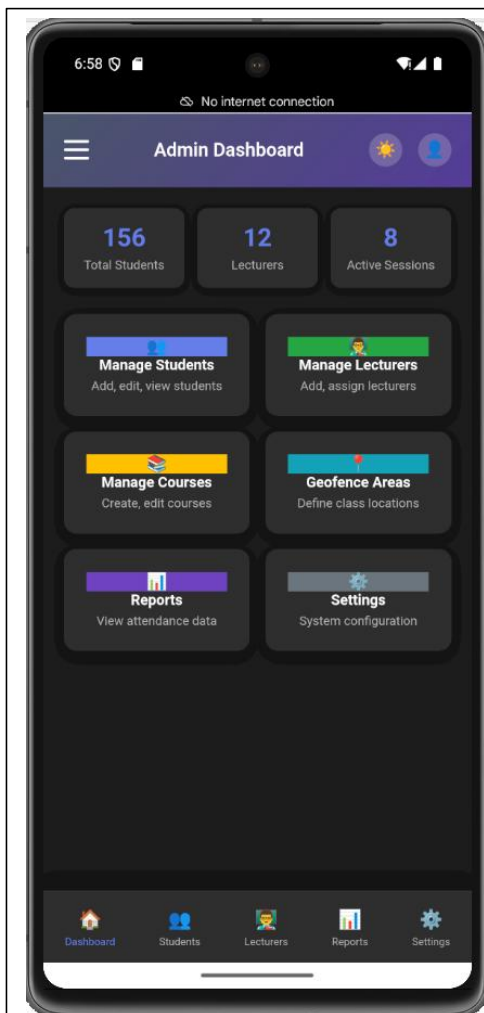
Testing and Deployment

The app was tested using an **Android emulator**, as shown in the screenshot. The `AndroidManifest.xml` file was configured to include necessary permissions for geofencing and camera access (for facial recognition). The emulator allowed for iterative testing of UI components, navigation, and simulated API calls.

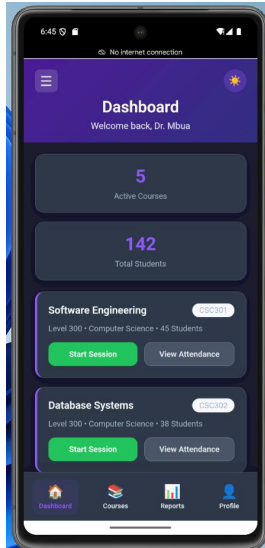
STUDENTS



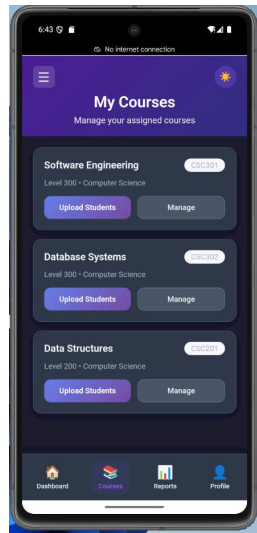
ADMINS



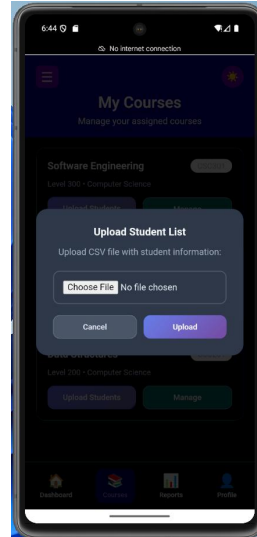
LECTURERS



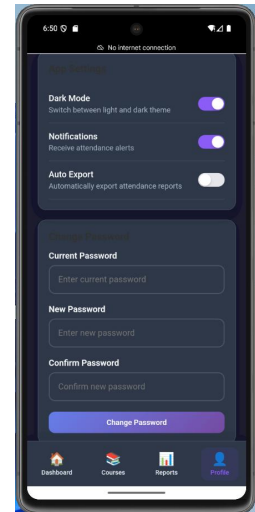
DASHBOARD



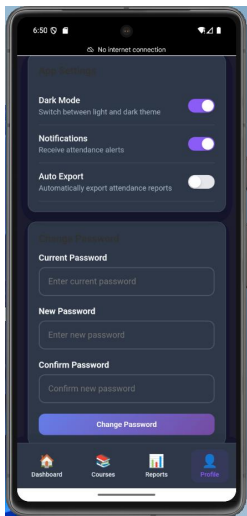
COURSE



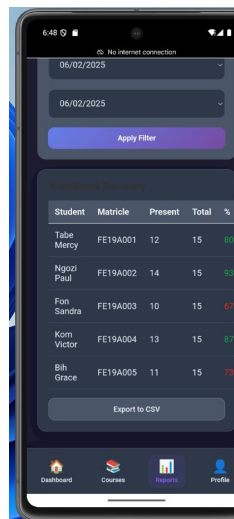
COURSE LIST UPLOAD



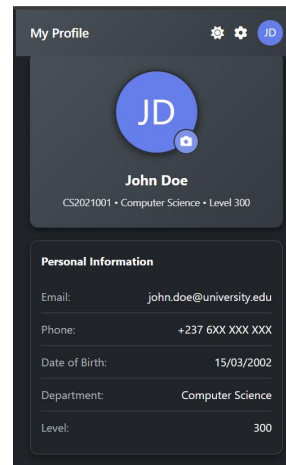
SETTINGS



SETTINGS

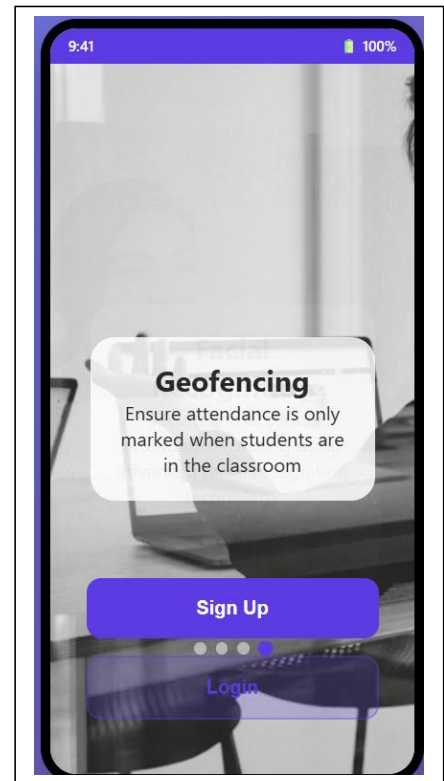
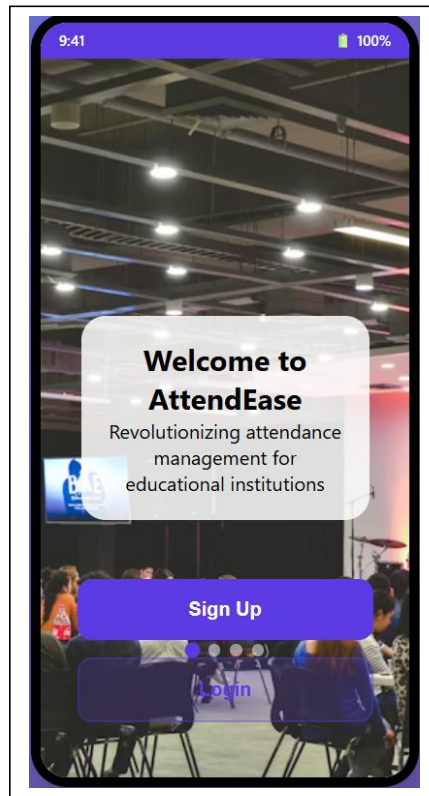
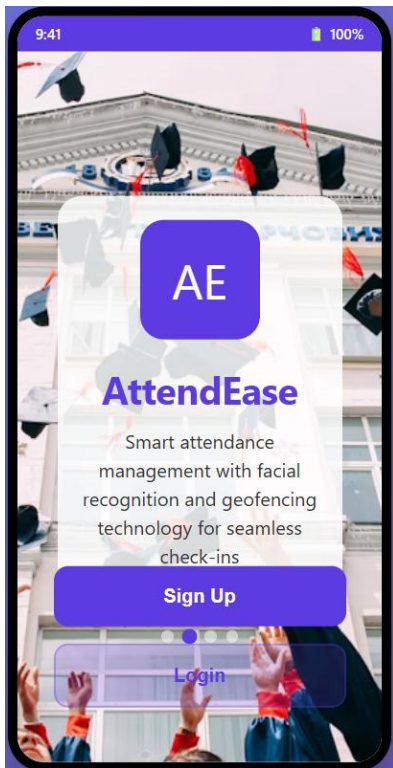


REPORT FORMAT

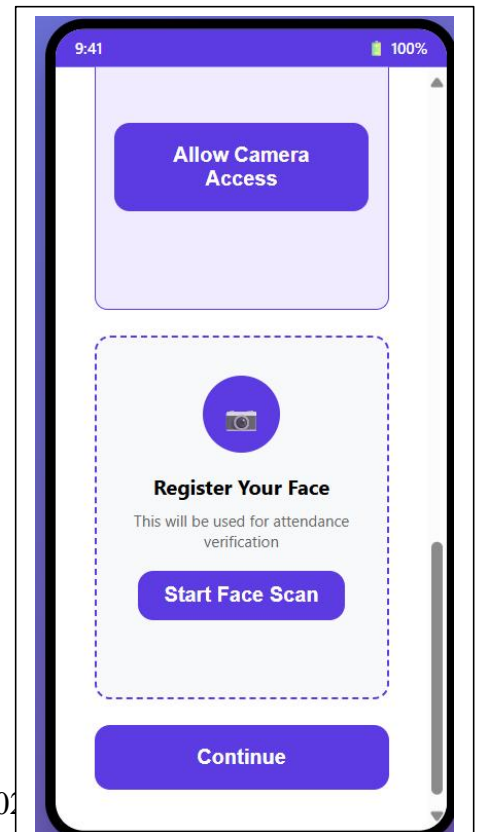
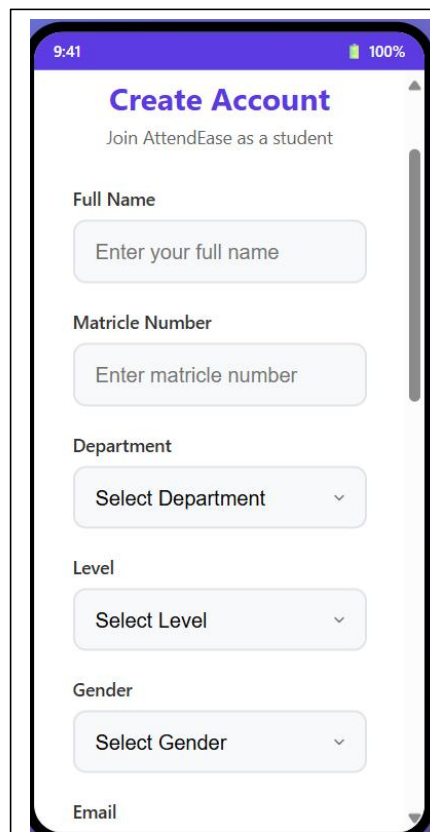
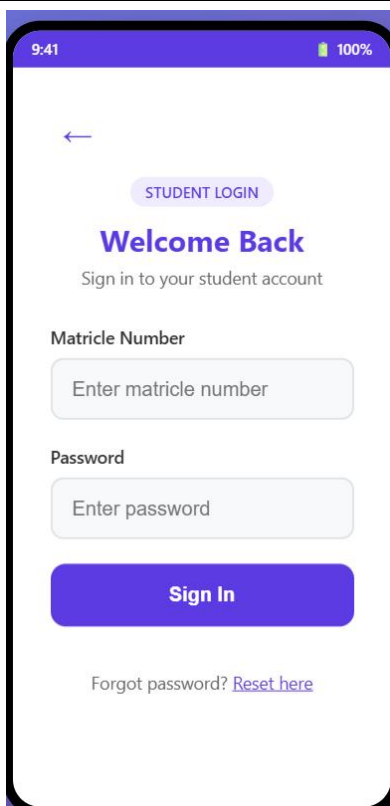


PROFILE

Home page and login (home transition between these)



Signin and login



Software Tools and Libraries

Software Tools and Their Functions

The following software tools were used in the development of AttendEase:

Software tools used in the AttendEase project.

Software	Function
Software	Function
Figma	Used for UI/UX design, creating high-fidelity mockups and prototypes for all screens.
Flutter	Framework for cross-platform app development, enabling a single codebase for Android and iOS.
Dart	Programming language used with Flutter to write the app's logic and UI components.
Visual Studio Code (VSCode)	Code editor for writing and debugging Flutter/Dart code, with support for Android emulator integration.
Android Emulator	Simulated Android device for testing the app's functionality and UI on various screen sizes.
Lottie	Library used for animations (e.g., success animation in success_screen.dart).

Library Rules and Usage Guidelines

The following libraries from pubspec.yaml have specific rules and guidelines:

- **cupertino_icons:** Use iOS-style icons only for Cupertino widgets; avoid mixing with Material Design unless explicitly designed.
- **google_fonts:** Import fonts dynamically; ensure network connectivity for initial load; cache fonts for offline use.
- **provider:** Manage state with ChangeNotifier; avoid direct state manipulation outside providers; use Consumer or Provider.of for reactivity.
- **geolocator:** Request location permissions at runtime; handle location service errors gracefully; use StreamSubscription for continuous updates.
- **google_ml_kit:** Initialize ML Kit before use; configure face detection with appropriate options; note that full facial recognition requires a backend API integration.

Conclusion

The AttendEase app successfully delivers a modern attendance management solution through its well-defined identity, thoughtful design, and robust frontend implementation. The branding, centered around geofencing and facial recognition, is reflected in the logo and color scheme, creating a cohesive user experience. Figma enabled a streamlined design process, while Flutter ensured efficient cross-platform development. The use of an Android emulator facilitated thorough testing, ensuring the app's readiness for deployment.

Future enhancements could include integrating actual API calls for authentication and attendance tracking, adding iOS-specific testing, and implementing advanced geofencing algorithms. The modular folder structure and reusable widgets provide a solid foundation for scalability and maintenance.

References

- Nielsen, J. (1994). "10 Usability Heuristics for User Interface Design." Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Flutter Documentation. (2025). "Flutter API Documentation." Available at: <https://api.flutter.dev/>
- Material Design Guidelines. (2025). "Material Design Principles." Available at: <https://material.io/design>